

PERSONAL EXPENSIVE TRACKER



A PROJECTWORK

***Submitted in fulfillment of the requirements for the award of
the degree of***

MASTER OF COMPUTER APPLICATIONS (MCA)

Submitted by

Harikrishnan PS(TTAXMCA009)

Guided By

Mrs. SUBHADRA B MENON

Centre for Computer Science and Information Technology

UNIVERSITY OF CALICUT

CENTRE FOR COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

THALIKKULAM, Thrissur-680569

2023-2025

UNIVERSITY OF CALICUT
CENTRE FOR COMPUTER SCIENCE AND INFORMATION TECHNOLOGY
THALIKKULAM -680569



CERTIFICATE

This is to certify that the project report entitled “**PERSONAL EXPENSIVE TRACKER**” is a bona fide record of the work done by
Harikrishnan PS(TTAXMCA009) towards the fulfillment of the requirements
for the award of the degree of **MASTER OF COMPUTER APPLICATIONS (MCA)** of **University of Calicut**.

Mrs. SUBHADRA B MENON
Project Guide
CCSIT,THALIKKULAM

Mrs. SUBHADRA B MENON
Associate Co-ordinator
CCSIT,THALIKKULAM

Internal Examiner

External Examiner

DECLARATION

I, **Harikrishnan PS**, hereby declare that the project work entitled “**PERSONAL EXPENSIVE TRACKER**” submitted to **University of Calicut** in fulfilment of the requirements for the award of the degree of **Master of Computer Applications(MCA)** is an original work done by me under the guidance of **Mrs.SUBHADRA B MENON,,** Associate Coordinator of CCSIT:
Thalikkulam,Thrissur

Place: Thalikkulam,Thrissur

Harikrishnan PS

Date:

ACKNOWLEDGEMENT

I wish to place on record our profound indebtedness and gratitude to all those who have contributed, directly or indirectly, to make this project work a success.

At the very first, we express our deep gratitude to **God** almighty, for lending our the talent to complete this project successfully.

I am thankful to **Mr. JISANTH M G**, Administrative Officer, **Mrs.SUBHADRA B MENON**, Associate Coordinator of CCSIT Thalikulam for having permitted us to undertake this project at Grapesgenix Technical Solutions Pvt Ltd Thrissur, I express our sincere thanks to **Mrs.SUBHADRA B MENON**, internal guide for her valuable advices and proper guidance.

I am extending my thanks to our **parents, teachers** and **friends** who have always stood with us and encouraged for our successful completion of project.

Harikrishnan PS

ABSTRACT

The Expense Tracker system is a comprehensive and user-friendly solution designed to assist individuals in managing their financial activities efficiently. This system enables users to record and categorize their income and expenses, providing detailed insights into their spending habits. It includes functionalities like income tracking, expense categorization, bill management, and real-time balance updates, ensuring accurate and up-to-date financial data.

Built using the Django framework and SQLite database, the system is secure, reliable, and efficient, offering a seamless experience to users. Key features include validation mechanisms, customizable categories, and data visualization through reports, enhancing the user's ability to make informed financial decisions. The system also supports functionalities like bill image uploads and automated reminders for payment due dates, further simplifying financial management.

With its modular design, the Expense Tracker ensures scalability and adaptability, catering to diverse user requirements. The system aims to address the challenges of traditional financial management methods by providing a digital, intuitive, and robust platform. This project demonstrates the potential of technology in promoting better financial awareness and decision-making, offering a valuable tool for modern-day users.

CONTENTS

No.	Section	Page No.
1	INTRODUCTION	01
1.1	System Overview	02
1.2	Objectives of the System	03
2	SYSTEM ANALYSIS	04
2.1	Existing System	05
2.2	Drawbacks of Existing System	05
2.3	Requirement Analysis	06
2.4	Proposed System	06
2.5	Advantages of Proposed System	08
2.6	Feasibility Study	08
3	SYSTEM SPECIFICATION	12
3.1	Hardware Specification	13
3.2	Software Specification	14
4	SYSTEM DESIGN	16
4.1	Input Design	20
4.2	Output Design	23
4.3	Database Design	24
4.4	Data Flow Diagram	33
5	SYSTEM TESTING	42
6	SYSTEM IMPLEMENTATION AND MAINTENANCE	47
7	CONCLUSION	49
8	SCOPE FOR FUTURE ENHANCEMENTS	52
9	APPENDIX	54
9.1	Database Design	55
9.2	Data Flow Design	59
9.3	ER Diagrams	62
9.4	Screen Layouts	63
10	BIBLIOGRAPHY	74

INTRODUCTION

1.INTRODUCTION

1.1 System Overview

The "Our Expense Tracker" mini project is a comprehensive personal finance management application, built with Django and SQLite3, that allows users to efficiently track their financial activities. The application includes modules for user registration, feedback submission, income and expense management, bill tracking, and payment status monitoring. These modules work together to offer users a streamlined experience for managing various financial transactions, enhancing their understanding of spending patterns, and ensuring timely bill payments. The system begins with user registration, where essential information, such as first name, last name, email, password, profile image, and phone number, is collected. This data is managed in the `users_register` model, ensuring each user has a unique profile and is associated with their specific income, expenses, bills, and feedback. A feedback module allows users to provide comments and ratings on their experience with the application. This feature helps administrators gather valuable insights into user satisfaction and potential areas for improvement. The core of the application lies in the income and expense tracking features. The `Income` model records user income sources, amounts, and dates, creating a log that users can refer to when analyzing their financial inflow. Similarly, the `Expense` model records user expenses, categorizing them by type, description, amount, and date. A validation feature ensures that only positive expense amounts are accepted, preventing data errors and ensuring accurate financial tracking. Another vital component is the `Bill` model, which helps users track regular bills such as electricity, water, rent, and more. This model enables users to store essential details like category, description, due date, amount, and payment status. Users can also upload images of their bills, providing an easy reference and record of past payments. The system supports partial integration with Razorpay, allowing users to track online payments for their bills through the generated Razorpay order ID. Finally, the application includes a `Payment` model that tracks individual payments made against bills. Each payment record includes the associated bill, payment ID, amount, currency, and success status, along with timestamps for when the payment was created and last updated. This feature ensures that users have a clear view of successful and failed payments, helping them manage and track their outstanding bills more effectively. Together, these components provide a holistic approach to personal finance management, making "Our Expense Tracker" an efficient tool for users to monitor, control, and improve their financial well-being.

1.2 Objectives of the system

Project Name: "Our Expense Tracker"

This project aims to provide an easy-to-use platform for managing personal finances, where users can log income, track expenses, manage bills, and monitor payments.

Objectives

- User Profile Management: Allow users to securely register and manage their profile for personalized tracking.
- Income Tracking: Record and categorize all income sources for comprehensive financial insight.
- Expense Logging: Track expenses with customizable categories to enhance budget planning.
- Bill Management: Manage recurring bills, set due dates, and mark payment status.
- Bill Image Storage: Enable users to upload and store images of bills as records.
- Payment Tracking: Monitor payment success/failure for bills, with Razorpay integration.
- Feedback Collection: Allow users to provide feedback for continuous improvement.
- Data Accuracy: Validate data entries to maintain precise income and expense records.
- Financial Insights: Offer a consolidated view of finances to improve financial planning.
- Budget Awareness: Help users stay informed of their spending and outstanding bills.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

The system analysis of "Our Expense Tracker" focuses on solving the challenges individuals face in managing their personal finances. Many people struggle with organizing income, tracking expenses, and paying bills on time, often relying on manual methods prone to errors. This project aims to provide a structured, user-friendly platform for logging financial data, tracking payment statuses, and offering insights to help users make better financial decisions. The system's requirements include features such as secure user registration, income and expense tracking with categories, bill management with due dates and statuses, and the ability to upload bill images for record-keeping. Payment tracking is integrated with Razorpay to ensure smooth and transparent online payment management. Additionally, the system collects user feedback to continuously improve functionality. Non-functional aspects like scalability, security, and ease of use are also prioritized to ensure the platform serves a broad audience effectively. To ensure feasibility, the system uses Django and SQLite3 for development, which are cost-effective and technically robust. Challenges like data validation, secure handling of sensitive information, and effective tracking of payments are addressed through Django's built-in security features and validation mechanisms. With this analysis, the project is designed to be a reliable, efficient, and scalable solution to personal finance management problems, catering to user needs comprehensively.

2.1 Existing System

The existing systems for managing personal finances are often manual or fragmented, making them inefficient and prone to errors. Many people rely on spreadsheets, paper records, or standalone apps that lack integration and do not provide a comprehensive view of their financial activities. These traditional methods make it challenging to track income, expenses, and bills simultaneously, leading to missed payments, untracked spending, and poor financial planning. Additionally, most systems lack features like bill image uploads, real-time payment status updates, and detailed categorization, which are essential for efficient financial management. As a result, users face difficulties in maintaining accurate records, generating insights, and making informed financial decisions. This project aims to address these limitations by providing an all-in-one, user-friendly solution for effective financial tracking and management.

2.2 Drawbacks of Existing System

- **Prone to Errors:** Manual tracking methods are error-prone and unreliable.
- **Lack of Integration:** Existing systems fail to combine income, expense, and bill tracking in one platform.
- **Missed Payments:** Absence of automated reminders leads to delayed or missed bill payments.
- **No Data Security:** Many methods lack secure handling of sensitive financial information.

2.3 Requirement Analysis

Requirement Analysis for the "Our Expense Tracker" project focuses on identifying the key features and technical requirements needed to meet user needs for managing their personal finances effectively. The system must allow users to securely register, log their income and expenses, categorize transactions, and track bill payments. Features such as the ability to upload bill images, real-time payment status tracking, and a feedback mechanism are essential for improving user experience. Additionally, the system should provide validation to ensure accurate financial data entry, preventing errors in income and expense tracking. A secure and scalable infrastructure is required to protect sensitive financial information while supporting growing numbers of users and transactions. The system also needs to be user-friendly and accessible across different devices. This includes having an intuitive interface for both mobile and desktop platforms. Integration with payment gateways like Razorpay is essential for tracking online bill payments, while real-time updates for bill statuses ensure users are always aware of their current financial obligations. The platform should also generate reports or insights, helping users analyze their spending habits and plan their finances better. Data security features, including encryption for sensitive details and secure login methods, are crucial to maintaining user trust and protecting personal financial data from unauthorized access.

2.4 Proposed System

The Expense Tracker is designed as a comprehensive solution to address the limitations of existing personal finance management systems. The proposed system aims to offer a seamless and integrated platform where users can easily track their income, expenses, and bills in one place. It will feature secure user registration, real-time payment status tracking, and the

ability to upload bill images for record-keeping. Users will also be able to categorize their expenses and income for better financial analysis, and automated reminders will help ensure bills are paid on time. The system will be built using Django and SQLite3 for a reliable and scalable backend, ensuring data security and smooth performance as the user base grows.

In addition to these core functionalities, the Expense Tracker will offer valuable insights through spending reports and graphical representations of financial data. Integration with payment gateways like Razorpay will allow users to track online bill payments directly within the app. The system will also have built-in data validation to prevent errors in financial entries and will ensure that all sensitive information is securely stored using encryption and secure login methods. This approach will provide users with a reliable, user-friendly tool to manage their finances more effectively and make informed decisions about their spending and savings.

Proposed system consists of 4 modules, they are:

- USER
- ADMIN

User Management Module:

- User registration and login
- Profile management
- Income and expense tracking
- Expense categorization
- Bill image uploads
- Due date tracking and payment status monitoring
- Data validation

Admin Management Module:

- User account management
- Feedback review
- Report generation
- Category and settings management
- System activity monitoring
- Security and data integrity maintenance

2.5 Advantages of the Expense Tracker System:

Centralized Financial Management: The system allows users to track all their income, expenses, and bills in one place, providing a comprehensive overview of their financial status.

Enhanced Data Accuracy: Built-in data validation ensures that only accurate financial entries are made, reducing human errors.

User-Friendly Interface: The system is designed to be intuitive and easy to use, making it accessible even for users with minimal technical knowledge.

Real-Time Tracking: Users can track expenses and bill payments in real-time, ensuring they stay on top of their finances and never miss a payment.

Automated Reminders: The system sends automated reminders for upcoming bills and payments, helping users avoid late fees and stay organized.

Financial Insights: The system provides detailed reports and spending analysis, helping users understand their financial habits and make better budgeting decisions.

Secure Data Storage: Sensitive financial information is securely stored, protecting user privacy and preventing unauthorized access.

Bill Image Storage: Users can upload and store bill images, keeping all their financial documents organized and accessible.

Scalability: The system is designed to accommodate an increasing number of users and data, ensuring long-term usability.

Feedback Integration: Users can provide feedback on the system, enabling continuous improvement and adaptation to their needs.

2.6 Feasibility Study

Technical Feasibility:

The Expense Tracker system will be developed using Django, a robust web framework that offers scalability, security, and flexibility. SQLite3 will be used for the database, ensuring reliable and efficient data storage. The system integrates with Razorpay for payment tracking, which is well-documented and widely used for secure online transactions. The proposed technology stack is capable of handling the system's requirements and is technically feasible within the project scope.

Economic Feasibility:

The use of open-source tools like Django and SQLite3 minimizes development and operational costs. There are no significant licensing fees involved in the chosen technology, making the system cost-effective. Additionally, the integration of payment gateways like Razorpay ensures that users can make secure payments without requiring custom payment infrastructure, further reducing costs. The system's benefits, including time-saving and accurate financial management, provide good value for money, even if the user base scales.

Operational Feasibility:

The Expense Tracker is designed to be user-friendly and easy to adopt, ensuring that individuals with little technical knowledge can use the system effectively. The system will be accessible via web browsers, making it widely available across different devices. Automated reminders and real-time tracking features ensure that users remain engaged and continue using the system. Continuous feedback collection will help improve the system, ensuring it meets user needs over time.

Legal and Security Feasibility:

The system will comply with data protection regulations by implementing secure user authentication and data encryption protocols. Sensitive financial information will be protected using best practices for data security. Legal requirements related to data storage and privacy will be considered during the development phase, ensuring that user information is handled responsibly and securely.

Economical Feasibility

The Expense Tracker system is designed to be cost-effective both in terms of development and long-term operation. The use of open-source technologies like Django and SQLite3 significantly reduces development costs as there are no licensing fees associated with these tools. Additionally, these technologies are widely supported, ensuring ease of maintenance and reducing the need for specialized expertise, which helps lower ongoing operational costs. The integration of payment gateways like Razorpay eliminates the need for building a custom payment processing system, which would have been both costly and time-consuming. Razorpay offers cost-effective transaction fees, and the system will allow users to make secure online payments without additional infrastructure costs. The system's design is scalable, meaning it can accommodate an increasing number of users and financial data without requiring significant infrastructure upgrades, which helps keep operational costs low.

With its automated features, such as reminders for bill payments, users will save time and avoid late fees, further adding to the system's economic value. Overall, the Expense Tracker system provides a cost-effective solution for both users and developers, with manageable initial development costs and low maintenance expenses, making it economically feasible for wide adoption.

Operational Feasibility

The Expense Tracker system is designed with ease of use and accessibility in mind, ensuring it meets the operational needs of its users. The system's user interface will be intuitive, making it simple for individuals with minimal technical knowledge to navigate and manage their finances effectively. Users will be able to register, log in, track their income and expenses, and monitor bill payments without requiring extensive training or technical expertise. This ensures high user adoption and minimizes the risk of user resistance. The system will be accessible via web browsers, making it available on various devices such as smartphones, tablets, and desktops. This broad accessibility enhances convenience and ensures that users can track their financial activities from anywhere at any time. Furthermore, automated reminders for bill payments and real-time expense tracking features will help keep users engaged and make the system more useful in their daily lives, increasing the likelihood of continued use. The Expense Tracker will also include a feedback mechanism, allowing users to provide input on their experience. This will enable continuous improvement of the system, addressing any operational challenges and adapting to user needs. With scalable infrastructure and the use of proven technologies, the system is capable of handling growth in the number of users and transactions, ensuring reliable performance over time. Overall, the Expense Tracker system is operationally feasible, offering a user-friendly, accessible, and efficient platform that meets the needs of its users while remaining adaptable and scalable for future growth.

Technical Feasibility

The Expense Tracker system is technically feasible, as it leverages proven technologies that are well-suited for the system's requirements. The backend will be built using Django, a widely used and robust web framework known for its scalability, security features, and rapid development capabilities. Django provides built-in tools for user authentication, data validation, and handling databases, making it ideal for developing a secure and efficient financial tracking system.

The system will use SQLite3 for database management, which is a lightweight and efficient database engine that works seamlessly with Django. SQLite3 is easy to set up and maintain, making it an excellent choice for the system, especially for initial development stages and small to medium-sized user bases. As the user base grows, the database can be easily migrated to a more scalable system, such as PostgreSQL, without significant changes to the core codebase. For payment processing, the system will integrate with Razorpay, a reliable and secure payment gateway. Razorpay offers simple API integration, ensuring that online bill payments can be processed seamlessly and securely within the system. This integration minimizes the need for developing a custom payment solution, reducing both development time and complexity. The system will also feature secure data handling through encryption and secure login mechanisms, ensuring that sensitive financial information is protected. The Expense Tracker will be accessible via web browsers, making it cross-platform and device-agnostic. The use of cloud hosting services will ensure scalability and reliability, enabling the system to handle an increasing number of users and transactions without performance issues. Overall, the technical feasibility of the Expense Tracker system is high, given the use of reliable technologies, secure practices, and scalable infrastructure that can support the system's growth and long-term sustainability.

SYSTEM SPECIFICATION

3.SYSTEM SPECIFICATION

3.1 Hardware Specification

The Expense Tracker system is a web-based application that primarily depends on server-side infrastructure for hosting and user access. However, there are minimum hardware specifications required for the development, testing, and deployment of the system:

Server Requirements (for Hosting the Application):

Processor: Dual-core processor (Intel i5 or equivalent) or better for handling multiple users.

RAM: Minimum 4 GB of RAM (8 GB recommended) to ensure smooth processing and handling of concurrent requests.

Storage: At least 100 GB of disk space for storing user data, images (bill uploads), logs, and backups. SSD storage is recommended for faster data access.

Network: Stable internet connection with at least 1 Mbps speed to ensure smooth data exchange between the server and users.

Backup System: Cloud backup solutions or on-site backup systems for secure storage of critical data, ensuring data integrity and recovery in case of failure.

Development and Testing Machine (for Developers and Testers):

Processor: Intel Core i5 or better (preferably 2.5 GHz or higher).

RAM: 8 GB or more for smooth development, testing, and running of local servers.

Storage: Minimum 100 GB of free disk space for local development, databases, and testing environments.

Network: Reliable internet connection for accessing development tools, testing APIs, and integrating third-party services (e.g., Razorpay).

Operating System: Windows 10/11, macOS, or Linux for cross-platform development compatibility.

End User Hardware (for Accessing the Application):

Devices: Desktop, laptop, tablet, or smartphone with internet access.

Processor: Any modern processor (Intel i3, ARM, etc.) sufficient for browsing.

RAM: 2 GB of RAM or more for smooth operation on devices accessing the web application.

Screen Resolution: Minimum 1024x768 resolution for clear display, though responsive design ensures compatibility with all screen sizes.

These hardware specifications are designed to ensure that the Expense Tracker system performs optimally during both development and usage, providing a smooth and reliable user experience.

3.2 Software Specification

The **Expense Tracker** system requires several software tools and technologies for development, deployment, and operation. Below are the key software specifications for the system:

1. Operating System:

- **Windows, macOS, or Linux** (for development and deployment environments)
- Web server and cloud hosting can be done on Linux-based servers (e.g., Ubuntu, CentOS) for better performance and stability.

2. Web Framework:

- **Django (Python)**: The core web framework for developing the system. It provides built-in features for user authentication, database management, and security, streamlining the development process.

3. Database Management System:

- **SQLite3**: A lightweight, serverless relational database for initial development. It is easy to set up and manage for small to medium-sized applications.
- **PostgreSQL or MySQL**: These can be used as alternatives for more scalability, especially when the user base increases.

4. Programming Languages:

- **Python**: For backend development using Django.
- **HTML5, CSS3, JavaScript**: For frontend development to create a responsive and user-friendly interface.
- **Bootstrap**: A frontend framework to ensure responsive design across devices.

5. Web Server:

- **Nginx or Apache**: These are popular web servers that can be used to serve the Django application in production.

- **Gunicorn:** A Python WSGI HTTP server commonly used with Django applications for better performance in production.

6. **Payment Gateway Integration:**

- **Razorpay API:** For handling secure payment transactions within the system.

7. **Version Control System:**

- **Git:** For source code management and version control, ensuring collaboration among development team members.
- **GitHub or GitLab:** For hosting and managing the Git repository.

8. **Cloud Hosting/Deployment:**

- **Heroku:** For simple deployment and hosting during development and testing.
- **AWS (Amazon Web Services) or DigitalOcean:** For scalable production hosting, database management, and storage solutions.

9. **Security:**

- **SSL Certificates:** For securing user data with HTTPS during data transmission.
- **Django Security Features:** Built-in protection for common web vulnerabilities such as Cross-Site Scripting (XSS), SQL injection, and Cross-Site Request Forgery (CSRF).

10. **Additional Tools:**

- **Celery (with Redis or RabbitMQ):** For handling background tasks, such as sending reminders for upcoming bill payments or processing long-running tasks.
- **Django REST Framework (optional):** If the system will offer API integration or mobile app support in the future.
- **Postman:** For testing APIs and ensuring that integration with third-party services (like Razorpay) works correctly.

These software specifications ensure that the **Expense Tracker** system is developed with robust, secure, and scalable technologies, providing a seamless user experience while maintaining performance and reliability.

SYSTEM DESIGN

4.SYSTEM DESIGN

The **Expense Tracker** system is designed to efficiently manage user finances, track income and expenses, and offer various reporting and notification features. Below is an overview of the system design:

Fundamental Design Concepts of System

The design of the **Expense Tracker** system is based on key fundamental design principles that ensure the system is efficient, scalable, secure, and user-friendly. These principles guide the overall architecture, modularization, and performance considerations. Below are the core design concepts:

1. Modularity

- The system is broken down into distinct, manageable modules, each handling a specific functionality. This allows for easier development, testing, and maintenance. Key modules include user management, income and expense tracking, bill management, and admin controls.
- Each module interacts with others through well-defined interfaces, ensuring that changes in one module do not affect others unnecessarily.

2. Scalability

- The system is designed to handle growth in both data volume (more transactions, users, bills) and user base. By using technologies like Django, which is scalable, and databases like SQLite (initially) and PostgreSQL (for future scaling), the system can easily accommodate a larger number of users and transactions.
- Future expansion (e.g., adding mobile apps or API integrations) is considered in the design, ensuring that the system can scale both in terms of functionality and performance.

3. Security

- Data security is a fundamental design concern, especially since the system will handle sensitive user information such as financial data, personal details, and payment information.

- The system employs secure authentication methods (password hashing, token-based login), ensures HTTPS for secure data transmission, and includes data validation to prevent invalid or malicious inputs.
- Payment processing integrates with a secure third-party provider (Razorpay), which ensures secure transactions.

4. Usability

- The system is designed to be intuitive and easy for users of all technical skill levels. Simple and user-friendly interfaces are a priority in both the frontend (user experience) and backend (admin interface).
- The system features automated reminders for bill payments, easy-to-understand financial reports, and streamlined expense tracking to enhance user engagement and simplify financial management.

5. Maintainability

- The system follows coding best practices and employs a clear and organized structure, making it easier to maintain, extend, and debug. The use of the MVC architecture ensures a clear separation between logic, presentation, and data storage.
- With version control (Git) and detailed documentation, new developers can easily understand and contribute to the system over time.

6. Efficiency

- The design ensures that the system can process transactions and generate reports in a timely manner, even with a growing number of users. Efficient database queries and caching mechanisms are used to reduce load times and improve the overall system performance.
- The system uses background task processing (Celery with Redis) to handle time-consuming operations, such as sending reminders and generating reports, without blocking the main application workflow.

7. Flexibility and Extensibility

- The **Expense Tracker** system is designed to be flexible, allowing for future enhancements and extensions, such as adding new categories for expenses, integrating new payment gateways, or offering mobile app support.
- The modular nature of the system also makes it easier to add or modify features, such as reporting tools, budgeting options, or additional payment methods, as user needs evolve.

8. Consistency

- Consistency is maintained across all modules in terms of design patterns, user interface elements, and data structure. Consistent input validation, error handling, and reporting ensure that users always know what to expect.
- The design emphasizes standard conventions for user interactions, making the system predictable and easier for users to navigate.

9. Responsiveness

- The user interface is designed to be responsive, ensuring that the system works seamlessly across various devices and screen sizes (e.g., mobile, tablet, desktop). This is critical to providing a consistent and accessible user experience on any device.
- Responsive design principles ensure that content adapts dynamically to different screen sizes, providing an optimal viewing experience.

10. Separation of Concerns

- The **Expense Tracker** system uses the **Model-View-Controller (MVC)** design pattern to ensure that different aspects of the application (data handling, user interface, business logic) are separated into distinct layers.
- This separation helps reduce the complexity of the system, improves code maintainability, and allows for better scalability and testing.

These fundamental design concepts ensure that the **Expense Tracker** system is efficient, user-friendly, and scalable, while also ensuring that security and maintainability are central priorities throughout the development and future enhancements.

4.1 Input Design

The **Input Design** is a crucial aspect of the **Expense Tracker** system, as it focuses on how users interact with the system to provide the necessary data. Proper input design ensures that the data entered by users is accurate, consistent, and efficient for processing. Below is an overview of the input design for the system:

1. User Registration and Login Inputs

- **Fields:**
 - **First Name** (Text input)
 - **Last Name** (Text input)
 - **Email ID** (Email input, validated for correct format)
 - **Password** (Password input, must meet security criteria)
 - **Confirm Password** (Password input, should match the password)
 - **Phone Number** (Numeric input, validated for valid length and format)
 - **Profile Image** (Image upload, optional)
- **Input Constraints:**
 - Email should be unique.
 - Password and confirm password must match.
 - Phone number should be numeric and match a valid phone number format.
- **Validation:** Real-time validation for email format, password strength, and phone number format.

2. Income and Expense Tracking Inputs

- **Income Input:**
 - **Source** (Text input, e.g., Salary, Freelance work)
 - **Amount** (Decimal input with a minimum value of 0)
 - **Date** (Date picker input)
- **Expense Input:**

- **Category** (Dropdown list: e.g., Groceries, Rent, Utilities)
- **Description** (Text input, optional)
- **Amount** (Decimal input with validation to ensure a positive value)
- **Date** (Date picker input)
- **Validation:**
 - Ensure that the amount entered is greater than zero for both income and expenses.
 - Ensure that the date is a valid calendar date.

3. Bill Management Inputs

- **Bill Input:**
 - **Category** (Dropdown list: e.g., Electricity, Water, Rent)
 - **Description** (Text input, optional)
 - **Amount** (Decimal input with validation)
 - **Due Date** (Date picker input)
 - **Bill Image** (File upload for bill images, optional)
- **Validation:**
 - Amount must be greater than zero.
 - Due date should be a valid future date.

4. Feedback Inputs

- **Rating** (Dropdown or radio buttons: 1 to 5 stars)
- **Feedback Text** (Text area for the user to provide comments or suggestions)
- **Email** (Email input field to track the user giving feedback)
- **Validation:**
 - Rating must be a valid choice (1 to 5).
 - Feedback text should not exceed a certain character limit.

5. Admin Inputs (for Admin to Manage Data)

- **User Management:**
 - Admin can update or deactivate user accounts. Inputs would include **First Name, Last Name, Email, Phone, and Profile Image**.
- **Expense and Income Review:**
 - Admin can manually review and approve or modify expenses and income. These inputs are editable for amounts, categories, and descriptions.
- **Bill Payments:**
 - Admin inputs to manually mark bills as paid or unpaid. This would include selecting the bill and confirming payment status.

6. General Input Considerations:

- **Consistency:** All input fields across the system will use consistent formats and labeling (e.g., fields are clearly labeled, date formats are consistent).
- **Error Handling:** Clear, user-friendly error messages will appear when invalid data is entered, guiding users to correct their input (e.g., "Please enter a valid email address").
- **Data Validation:** Real-time validation checks for all fields to ensure data integrity (e.g., checking for empty fields, numeric inputs, valid date formats).
- **Accessibility:** Input fields will be designed to be accessible to users with disabilities, with appropriate labeling and keyboard navigation options.

7. Input Storage and Processing:

- All user input will be captured through form submissions, and once validated, data will be stored in the backend database (e.g., SQLite3 for development, PostgreSQL for production).
- For uploaded files (e.g., bill images), the system will handle file storage in a secure folder with necessary file type and size restrictions.

By ensuring a clear, structured, and intuitive **Input Design**, the **Expense Tracker** system allows users to efficiently input their financial information while minimizing errors and ensuring data quality.

4.2 Output Design

The **Output Design** refers to how data is presented to the user after it has been processed by the system. In the **Expense Tracker** system, output is generated in the form of reports, notifications, dashboards, and summaries to provide the user with clear, actionable insights into their financial activities. Proper output design ensures that the information is displayed in a user-friendly, organized, and meaningful way.

1. Dashboard Output

- **Purpose:** To provide users with an overview of their financial activities.
- **Output Elements:**
 - **Total Income:** A summary of all income entries for a specified period (e.g., current month).
 - **Total Expenses:** A summary of all expenses for the same period.
 - **Total Balance:** The difference between income and expenses, showing the available balance.
 - **Category-wise Expense Breakdown:** A pie chart or bar graph displaying how expenses are distributed across categories (e.g., Rent, Utilities, Groceries).
 - **Upcoming Bills:** A list of bills with their due dates and amounts, indicating any unpaid bills.
 - **Recent Transactions:** A summary of the latest income and expense transactions, showing the amount, category, and date.
- **Design Considerations:**
 - The dashboard provides at-a-glance data and uses visualizations (charts, graphs) for easy comprehension.
 - All financial data is updated in real-time based on the user's input.

2. Income and Expense Reports

- **Purpose:** To provide detailed reports on the user's income and expenses.
- **Output Elements:**
 - **Income Report:**
 - Breakdown of income sources (e.g., salary, freelance, gifts).
 - Total income for a selected period (daily, weekly, monthly, yearly).
 - Visual representation (bar chart, line graph) showing income trends over time.
 - **Expense Report:**
 - Breakdown of expenses by category (e.g., rent, utilities, entertainment).
 - Total expenses for the selected period.
 - Visual representation (pie chart, bar graph) showing the share of each category in total expenses.
- **Design Considerations:**
 - The user can filter and select the date range for the report.
 - Reports are presented in a tabular format for easy readability, with options for exporting (e.g., to PDF or Excel).
 - Graphs/charts are used to visualize trends, helping users quickly analyze their financial behavior.

3. Bill Payment Summary

- **Purpose:** To display the details of bills due for payment, payment status, and payment history.
- **Output Elements:**
 - **List of Bills:** Displays all bills with relevant details, such as category, amount, due date, and payment status (Paid/Unpaid).
 - **Bill Payment Status:** Clear indication of whether each bill is paid or pending.

- **Payment History:** A history of past payments with the payment date, amount, and transaction ID (if available).
- **Design Considerations:**
 - Bills are organized by due date, with the most urgent bills appearing at the top.
 - Users can see upcoming bills and easily mark bills as paid when making a payment.
 - Each bill has an option to view or upload the bill image for reference.

4. Financial Summary Report

- **Purpose:** To provide a holistic overview of the user's financial status over a period of time.
- **Output Elements:**
 - **Total Income vs. Total Expenses:** A comparative report showing income against expenses for the selected period.
 - **Balance Overview:** Shows the overall balance (income minus expenses).
 - **Savings vs. Spending:** A report showing the proportion of income that was saved versus the amount spent.
 - **Trends Analysis:** Insights into user spending and saving habits over time.
- **Design Considerations:**
 - The summary uses simple bar graphs or line graphs to show trends, making it easier for the user to understand the data.
 - Data is shown for selected date ranges (e.g., week, month, year).

5. Notifications and Alerts

- **Purpose:** To notify users of important events or actions related to their financial activities.
- **Output Elements:**
 - **Bill Payment Reminders:** Notifications for upcoming bill due dates.

- **Overdue Payments:** Alerts for any bills that are past their due date.
- **Budget Exceeding Notifications:** Alerts when expenses exceed the user-defined budget for a category.
- **Design Considerations:**
 - Notifications are displayed in real-time and are accessible via a notification center within the system.
 - Clear and actionable notifications with clickable links to the relevant bill or expense entry.

6. User Profile Output

- **Purpose:** To display the user's personal information and account details.
- **Output Elements:**
 - **Personal Details:** Name, email, phone number, profile picture.
 - **Account Settings:** Options to update personal information, change password, or delete account.
- **Design Considerations:**
 - Information is displayed clearly and with an option to edit the user details.
 - Security features like password management are easy to find and update.

7. Feedback and Rating

- **Purpose:** To display feedback from users after they have interacted with the system.
- **Output Elements:**
 - **User Ratings:** Display the ratings provided by users for their satisfaction with the system.
 - **Feedback Comments:** Show the text comments submitted by users.
 - **Average Rating:** Display the average rating for the system as a whole.
- **Design Considerations:**
 - Ratings are presented with stars (1 to 5) for easy visual understanding.

- Feedback comments are shown in a list with the user's email and the date of submission.

8. Export Options

- **Purpose:** To allow users to export financial data for external use.
- **Output Elements:**
 - **Export Income Report** (PDF, CSV, or Excel)
 - **Export Expense Report** (PDF, CSV, or Excel)
 - **Export Bill History** (PDF, CSV, or Excel)
- **Design Considerations:**
 - Export options are easy to access on each report or data page.
 - Files are formatted properly, with clear headings and organized data for the user.

Design Considerations for Output:

- **Clarity and Readability:** Outputs are designed to be simple and easy to understand, with the most important information displayed prominently. Clear labels, headings, and formatting are used to enhance readability.
- **User Interactivity:** Outputs allow users to take actions, such as filtering reports, exporting data, or marking bills as paid.
- **Data Visualization:** Graphs, charts, and tables are used to represent financial data in a way that allows users to easily analyze their spending and income trends.

By focusing on clear, organized, and interactive outputs, the **Expense Tracker** system provides users with valuable insights into their financial situation, helping them to make informed decisions and better manage their money.

4.3 Database Design

The **Database Design** is an essential part of any application that involves data storage and management. It defines the structure of the data, how it is stored, and how it can be accessed and manipulated. In the **Expense Tracker System**, the database is designed to efficiently

handle users' financial information, including income, expenses, bills, payments, and user feedback. The system uses a **relational database** (like **SQLite3** in this case, as the project uses Django) to store structured data in tables, with relationships between tables to maintain data integrity.

Database Structure Overview

The main entities in the system are **Users**, **Income**, **Expenses**, **Bills**, **Payments**, and **Feedback**. These entities will be represented as tables in the database, each with its own set of attributes (columns), and will be related to each other through **foreign keys** to ensure referential integrity. Below is an overview of the database design:

1. Users Table

Table Name: users_register

- **Attributes (Columns):**
 - **id** (Primary Key, Auto-incremented)
 - **first_name** (String, max length 100)
 - **last_name** (String, max length 100)
 - **email_id** (String, unique, max length 100)
 - **password** (String, max length 100)
 - **confirm_password** (String, max length 100)
 - **image** (Image field, nullable)
 - **phone** (Integer, nullable)
 - **Purpose:** This table stores the personal details of the users, including their name, email, password, profile image, and contact information (phone number).
-

2. Income Table

Table Name: Income

- **Attributes (Columns):**

- **id** (Primary Key, Auto-incremented)
 - **user_id** (Foreign Key referencing users_register.id)
 - **source** (String, max length 100)
 - **amount** (Decimal, max 10 digits with 2 decimal places)
 - **date** (Date)
 - **Purpose:** This table stores users' income information. It records the source of income, the amount, and the date of the transaction. It is linked to the users_register table via the user_id field.
-

3. Expenses Table

Table Name: Expense

- **Attributes (Columns):**
 - **id** (Primary Key, Auto-incremented)
 - **user_id** (Foreign Key referencing users_register.id)
 - **category** (String, max length 150)
 - **description** (Text, nullable)
 - **amount** (Decimal, max 10 digits with 2 decimal places)
 - **date** (Date, default to current date)
 - **Purpose:** This table stores the expenses recorded by the user, including the category of the expense (e.g., Rent, Groceries), the amount spent, and the date of the transaction.
-

4. Bills Table

Table Name: Bill

- **Attributes (Columns):**

- **id** (Primary Key, Auto-incremented)
 - **user_id** (Foreign Key referencing users_register.id)
 - **category** (String, max length 50, with options: 'Electricity', 'Water', 'Internet', 'Rent', 'Other')
 - **description** (Text, nullable)
 - **amount** (Decimal, max 10 digits with 2 decimal places)
 - **due_date** (Date)
 - **created_at** (DateTime, auto-set on creation)
 - **image** (Image field, nullable)
 - **is_paid** (Boolean, default to False)
 - **razorpay_order_id** (String, nullable)
 - **Purpose:** This table holds the bill information for users, including the bill category (e.g., Electricity, Water), the amount, due date, and payment status (is_paid). It is linked to the user via user_id and tracks bills with an optional image attachment (e.g., a scanned copy of the bill).
-

5. Payments Table

Table Name: Payment

- **Attributes (Columns):**
 - **id** (Primary Key, Auto-incremented)
 - **bill_id** (Foreign Key referencing Bill.id)
 - **payment_id** (String, max length 255)
 - **amount** (Decimal, max 10 digits with 2 decimal places)
 - **currency** (String, default 'INR')
 - **is_successful** (Boolean, default False)

- **created_at** (DateTime, auto-set on creation)
 - **updated_at** (DateTime, auto-set on update)
 - **Purpose:** This table stores the payment details for bills, including payment status (successful/failed), payment ID, amount, and currency. It is linked to the Bill table through `bill_id`.
-

6. Feedback Table

Table Name: Feedback

- **Attributes (Columns):**
 - **id** (Primary Key, Auto-incremented)
 - **feedback_text** (Text)
 - **rating** (Integer, choices from 1 to 5)
 - **created_at** (DateTime, auto-set on creation)
 - **email** (Email, max length 100)
 - **Purpose:** This table stores user feedback and ratings for the system. It captures the user's feedback text and the rating given (1 to 5), linked to their email.
-

Relationships Between Tables

1. **Users and Income:**
 - A one-to-many relationship between `users_register` and `Income`. A user can have multiple income sources.
2. **Users and Expenses:**
 - A one-to-many relationship between `users_register` and `Expense`. A user can record multiple expenses.
3. **Users and Bills:**

- A one-to-many relationship between users_register and Bill. A user can have multiple bills.

4. Bills and Payments:

- A one-to-many relationship between Bill and Payment. A bill can have multiple payments associated with it, e.g., multiple installment payments.

5. Users and Feedback:

- A one-to-many relationship between users_register and Feedback. A user can provide multiple feedback entries.

Entity-Relationship Diagram (ERD)

The ERD provides a visual representation of how these entities are related. The key entities (users_register, Income, Expense, Bill, Payment, Feedback) are connected by **foreign keys** that create relationships between them, ensuring referential integrity in the database.

Normalization

The database schema follows the principles of **normalization** to ensure that:

- Data redundancy is minimized.
- Data integrity is maintained.
- Relationships between entities are clear and logically structured.

For example:

- User information is stored in the users_register table and not duplicated in other tables.
- Income and expenses are kept in separate tables, which makes it easier to maintain and query data.

Conclusion

The **Expense Tracker System's database design** focuses on structuring financial data in a clear and efficient manner, ensuring quick access, easy manipulation, and data integrity. Each table is designed to capture the necessary details while establishing meaningful relationships between entities, allowing users to seamlessly track their financial activities, manage income and expenses, and make payments for bills.

Normalization

Normalization is the process of organizing the attributes (columns) and relations (tables) of a database to minimize redundancy and dependency. The goal of normalization is to ensure that the database is efficient, logically structured, and easy to maintain. It helps in eliminating redundant data, preventing data anomalies (insertion, update, and deletion anomalies), and improving the integrity of the data.

Normalization divides a database into two or more tables and defines relationships between the tables. The process involves applying a series of normal forms (NF) to the database schema, each with specific rules to ensure data consistency and integrity.

1. First Normal Form (1NF)

A table is in First Normal Form (1NF) if:

- It only contains atomic (indivisible) values, meaning there are no repeating groups or arrays.
- Each column contains only a single value, and each row is uniquely identifiable.

For example, in the `users_register` table:

- Each user should have only one value for `first_name`, `last_name`, `email_id`, etc.
- There should be no multi-valued attributes (e.g., multiple phone numbers in a single column).

2. Second Normal Form (2NF)

A table is in Second Normal Form (2NF) if:

- It is in 1NF.
- All non-key attributes are fully functionally dependent on the primary key.

In other words, there should be no partial dependency, meaning no non-key attribute depends only on part of the primary key (in case of a composite key).

For example, if a table has a composite primary key (e.g., `user_id` and `bill_id`), every non-key attribute must depend on both keys. If description only depends on `bill_id` and not on `user_id`, it would violate 2NF, and the table should be split into two.

3. Third Normal Form (3NF)

A table is in Third Normal Form (3NF) if:

- It is in 2NF.
- There are no transitive dependencies, meaning non-key attributes should not depend on other non-key attributes.

For example, if you have a table with columns `user_id`, `user_name`, `user_phone`, and `user_address`, and `user_phone` depends on `user_name` rather than `user_id`, this violates 3NF.

To fix it, the `user_name` and `user_phone` should be separated into a different table, leaving the `user_id` as the unique identifier in the original table.

4. Boyce-Codd Normal Form (BCNF)

A table is in Boyce-Codd Normal Form (BCNF) if:

- It is in 3NF.
- For every functional dependency ($X \rightarrow Y$), X is a superkey.

In simpler terms, BCNF ensures that every determinant in a table is a candidate key. BCNF is a stricter version of 3NF.

5. Fourth Normal Form (4NF)

A table is in Fourth Normal Form (4NF) if:

- It is in BCNF.
- Multi-valued dependencies are eliminated.

For example, if you have a table where each row can store multiple values for both hobbies and skills, this violates 4NF. To achieve 4NF, you would separate these into distinct tables.

6. Fifth Normal Form (5NF)

A table is in Fifth Normal Form (5NF) if:

- It is in 4NF.
- It cannot be decomposed into any smaller tables without losing information.

This is typically used when dealing with complex relationships where a single table cannot efficiently capture all the necessary details without redundancy.

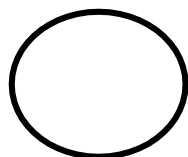
4.4 Data flow diagram symbols: -



- **Source or Destination of data**



Data Flow



Process



- **Storage**

Steps to Construct Data Flow Diagrams:

Four steps are commonly used to construct a DFD

- Process should be named and numbered for easy reference. Each name should be representative of the process.
- The destination of flow is from top to bottom and from left to right.
- When a process is exploded into lower-level details they are numbered.
- The names of data stores, sources and destinations are written in capital letters.

Rules for constructing a Data Flow Diagram:

- Arrows should not cross each other
- Squares, circles and files must bear names.
- Decomposed data flow squares and circles can have same names
- Draw all data flow around the outside of the diagram.

Context Diagram

The context diagram is the most abstract data flow representation of a system. It represents the entire system as a single bubble. This bubble is labeled according to the main function of the system. The various external entities with the system interact and the data flows occurring between the system and the external entities are also represented. The data input to the system and the data output from the system are represented as incoming and outgoing arrows. The name context diagram is well justified because it represents the context in which the system is to exist, i.e. the external entities who would interact with the system and the specific data items they would be supplying to the system and the data items they would be receiving from the system. The context diagram is also called the Level 0 DFD. The proposed system contains up to Level 1 DFD.

The levels in the proposed system are:

Context diagram: Data flow diagram (Level 0)

Admin: Data flow diagram (Level 1)

User: Data flow diagram (Level 1)

4.5 Architectural Diagram

An architecture diagram provides a visual representation of a system's structure, highlighting its components and their interactions. These diagrams are essential for understanding how different parts of a system collaborate to achieve its functionality. Typically, an architecture diagram will include components such as user interfaces, servers, databases, and external services. Each component is represented as a block or symbol, with lines or arrows depicting the relationships and data flows between them. For instance, in a web application architecture, the user interface (UI) interacts with the web server, which processes incoming requests. The web server then communicates with the application server, where the core business logic resides. The application server, in turn, accesses the database to retrieve or store data as needed. This hierarchical and interconnected layout helps stakeholders, from developers to business analysts, to grasp the system's overall structure quickly. By providing a clear and organized view, architecture diagrams facilitate better communication, planning, and decision-making throughout the development lifecycle.

4.6 Unified Modelling Language Design

The relation upon the system is structure through a conceptual ER-Diagram, which not only specifies the existential entities but also the standard relations through which the system exists and the cardinalities that are necessary for the system state to continue.

The entity Relationship Diagram (ERD) depicts the relationship between the data objects. The ERD is the notation that is used to conduct the data modeling activity the attributes of each data object noted is the ERD can be described resign a data object descriptions.

The set of primary components that are identified by the ERD are

- Data object
- Relationships
- Attributes
- Various types of indicators.

The primary purpose of the ERD is to represent data objects and their relationships.

4.7 Code design

The code design for the Expense Tracker System focuses on creating a modular, scalable, and efficient structure. This design involves organizing the code into logical components for each module, defining the relationships between models, views, and templates, and ensuring proper data flow throughout the application.

Key Components of Code Design

1. Modular Structure

The system is divided into distinct modules:

- **User Management Module:** Handles user registration, login, and profile management.
- **Expense Management Module:** Tracks user expenses and categories.
- **Income Management Module:** Tracks user income.
- **Bill Management Module:** Handles bill tracking, categorization, and payment status.
- **Admin Module:** Oversees data management and generates reports.

Each module has:

- **Models:** Represent database tables.
- **Views:** Define the logic for handling requests and responses.
- **Templates:** Render dynamic HTML pages.
- **Forms:** Manage user inputs.

Django Framework

The **Expense Tracker** built using the Django framework provides a robust, scalable, and efficient system for tracking and managing personal or organizational expenses. Below is an overview of how this system is structured using Django.

1. Framework Features

Django simplifies development with its built-in features:

- **MVC Pattern:** Django follows the Model-View-Template (MVT) architecture.

- **Built-in ORM:** Simplifies database interactions.
- **Admin Interface:** Automatically generated admin panel for managing data.
- **Authentication:** Handles user registration, login, and permissions.
- **Security:** Built-in protection against SQL injection, CSRF, XSS, etc

Key Modules in the Expense Tracker

a. User Module

- **Features:**
 - Register/Login using Django Authentication.
 - Manage user profile with fields like name, email, phone number, and profile picture.

b. Expense Management

- **Features:**
 - Add, view, edit, and delete expenses.
 - Categorize expenses (e.g., Food, Travel, Utilities).
 - View expenses filtered by date, category, or amount.

c. Income Management

- **Features:**
 - Track sources of income.
 - Add, edit, or delete income entries.
 - Generate summaries by month or year.

d. Reporting and Analysis

- **Features:**
 - Display income and expense summaries.
 - Visualize data using charts (e.g., Pie charts, Bar graphs with Django Chart.js).
 - Show remaining balance (Income - Expenses).
 -

e. Admin Module

- **Features:**
 - Manage user data and system records.
 - Generate reports on user activities.

SQLite3 Database

SQLite3 is an ideal choice for managing the database in a lightweight **Expense Tracker** system. It is a relational database engine that is easy to use, serverless, and integrated seamlessly with Django.

1. Key Features of SQLite3 in Expense Tracker

- **Lightweight and Serverless:** No additional setup is required.
- **Integrated with Django:** By default, Django uses SQLite3 as its database.
- **ACID-Compliant:** Ensures data reliability and transaction safety.
- **Supports Relational Models:** Handles tables, relationships, and queries efficiently.

VISUAL STUDIO 2022

Visual Studio 2022 is the latest release of Microsoft's integrated development environment (IDE), designed to provide a comprehensive suite of tools for developers to build applications for a wide range of platforms.

Visual Studio 2022 is a powerful, feature-rich IDE that supports development for desktop, web, cloud, mobile, gaming, and more. It offers a highly productive development environment with enhanced performance, better collaboration tools, and new features aimed at improving developer efficiency and code quality.

Key Features

- Enhanced Performance and 64-bit Architecture

- Improved Debugging and Diagnostics
- Productivity Enhancements
- Collaboration Tools
- New Project Templates and Workloads
- Accessibility Improvements
- Extension Ecosystem

.

SYSTEM TESTING

5.System Testing

System testing is a critical aspect of Software Quality Assurance and represents the ultimate review of specification, design and coding. Testing is a process of executing a program with the intent of finding an error. A good test is one that has a probability of finding an as yet undiscovered error. The purpose of testing is to identify and correct bugs in the developed system. Nothing is complete without testing. Testing is the vital to the success of the system.

The entire testing process can be divided into different phases:

- Unit testing
- Integration testing
- Validation testing
- Acceptance testing

5.1 Unit Testing

The first level of testing is unit testing. This test focuses on each module like admin, tutor, seller, customer. This is also known as “Module Testing”. To check whether each module in the software is proper and it gives desired outputs to the given inputs. All validations and conditions are tested in the module in the first unit. The goal is to test the internal logic of the module. In unit testing each unit are tested during the programming stage itself, in the proposed system we tested the following.

- The insertion of values in the database.
- The updating of values in to the database.
- The deletion of values from the database.
- Response of the system to inputs
- All the needed forms are checked for validation.

- The correctness of the phone number format (10-digit number).
- Whether the entries are in correct format.
- Email validation (email address must contain at least @ sign and a dot(.))
- Characters or numbers are within the limit given (proper working of all fields, response of the system is delaying or not etc).

5.2 Integration Testing

Integration testing ensures that the various modules of the Expense Tracker system work together as a complete software package. This testing phase addresses issues related to the verification of interactions and dependencies among integrated components. The goal is to validate the functional, performance, and reliability requirements of the system after all the modules are combined.

In the Expense Tracker system, individual modules such as User Management, Expense Management, Income Management, and Admin Dashboard are integrated and tested as a unified whole. During this process, the following aspects are tested:

System Response Time: Checked if the system responds promptly after integrating modules.

Page Linking: Verified that navigation and linking between all pages are working correctly.

Database Connectivity:

Validated the insertion, updating, and deletion of data within the system.

Ensured seamless data flow between the database and the integrated modules.

Functional Testing:

Confirmed that users can add, view, and manage expenses and incomes.

Ensured admin functionalities, such as managing categories, user roles, and reports, work properly.

Overall System Functionality:

Ensured all features, such as generating reports, calculating balances, and user authentication, work as expected after integration.

5.3 Validation Testing

Validation Testing is the process of evaluating software during the development process or at the end of the development process to determine whether it satisfies specified business requirements. Validation Testing ensures that the product actually meets the client's needs.

Validation testing can be defined in many ways, but a simple definition is that validation succeeds when the software functions in a manner that can reasonable expected by the user. That means the input we are entering to the system “PERSONAL EXPENSIVE TRACKER” will be checked whether the entered input is valid or not.

5.4 Acceptance Testing

This is the key factor for the success of any system. User acceptance is done by constantly keeping in touch with prospective system at the time of development and making changes whenever required. This is done with regard to the input and output screen designs.

The Different Test Cases are

- Guarantee that all independent parts within a module have been exercised at least once. In PERSONAL EXPENSIVE TRACKER the independent Parts of the system Tested and it fulfil the acceptance of the user requirements.
- Exercise all logical design on their true/false side
- Exercise all loops at their boundaries and within their operational bounds
- Each module is tested and modules are linked and integration is carried out

Levels of Testing

Acceptance Testing : Client Needs System Testing

Requirements Integration Testing : Design

Unit Testing : Code

Testing for the Proposed System

Each module in the proposed system undergone unit testing that evaluated each module of the system. When all the forms provided exactly correct output each of them are integrated to

form a particular module. Necessary validation including numeric validation, string validation, email validation and mandatory field validation are done to ensure reliability of output. Thus, the proposed system “PERSONAL EXPENSIVE TRACKER” covered all the major phases of testing. It proves to be a system that functions according to requirements.

SYSTEM IMPLEMENTATION AND MAINTENANCE

6. SYSTEM IMPLEMENTATION AND MAINTENANCE

6.1 Implementation

Implementation includes all those activities that take place to convert from the old system to the new. The old system consists of manual operations, which is operated in a very different manner from the proposed new system. A proper implementation is essential to provide a reliable system to meet the requirements of the organizations. An improper installation may affect the success of the computerized system.

Implementation Plan

The implementation plan includes a description of all the activities that must occur to implement the new system and to put into operation. It identifies the personnel responsible for the activities and prepares a time chart for implementing the system. The implementation plan consists of the following steps:

- List all files required for implementation.
- Identify all data required to build new files during the implementation.
- List all new documents and procedures that go into the new system.

The implementation plan should anticipate possible problems and must be able to deal with them. The usual problems may be missing documents; mixed data formats between current and files, errors in data translation, missing data etc. So, the proposed system satisfies all the implementation plan properly.

6.2 Maintenance

Software maintenance is the process of modifying a software system or component after its delivery in order to correct faults, improve the performance and the other attributes, or to adapt to the changed environment. Maintenance covers a wide range of activities including correcting the coding and design errors, updating the documentation and test data, and upgrading the user support. There is an aging process that calls for periodic maintenance of hardware software. Maintenance is always necessary to keep the software usable and useful. Hardware also requires periodic maintenance activities can be classified into:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance
- Preventive Maintenance

Corrective Maintenance: -

Even with the best quality assurance activities, it is likely that the customer will uncover defects in the software. Corrective maintenance changes the software to correct defects. Here the customers can know about the correct defects.

Adaptive Maintenance: -

Over time, the original environment (e.g. CPU, Operating System, Business Rules, External Product Characteristics) for which is developed is likely to change. Adaptive maintenance results in the modification to the software to accommodate changes to its external environment.

Perfective Maintenance: -

Modification of a software product after delivery to improve performance or maintainability.

Preventive Maintenance: -

Computer software deteriorates due to change, and because of this, preventive maintenance, often called software re-engineering, must be conducted to enable preventive maintenance makes changes to computer programs. So that they can be more easily corrected, adapted and enhanced.

CONCLUSION

7. Conclusion

The **Expense Tracker** system is a comprehensive solution for individuals seeking to manage their finances effectively. It provides a centralized platform to track income and expenses, helping users stay organized and in control of their financial activities. With features like categorizing transactions, real-time balance updates, and detailed reports, it empowers users to monitor their spending habits and make informed financial decisions.

This system is designed with a user-friendly interface to ensure ease of use for people with varying technical expertise. Its robust backend, powered by the Django framework and SQLite database, guarantees secure and efficient data handling. Validations, such as mandatory field checks and numeric constraints, ensure the reliability of inputs and outputs, reducing errors and maintaining data integrity.

By integrating modules seamlessly, the **Expense Tracker** delivers a cohesive experience where users can perform various operations—such as adding, editing, or deleting records—with minimal effort. Its ability to provide accurate, up-to-date financial summaries makes it an invaluable tool for personal financial planning. Overall, the **Expense Tracker** meets the critical need for a modern, reliable, and efficient financial management solution, proving to be a highly beneficial system for its users.

SCOPE FOR FUTURE ENHANCEMENTS

8. Scope for Future Enhancements

The **Expense Tracker** system has significant potential for future upgrades, aiming to make financial management even more seamless and efficient. One key enhancement could be the introduction of multi-currency support, enabling users to track expenses and incomes in different currencies with real-time conversion rates. Integrating with banking systems could further automate the process by syncing transaction data directly, reducing manual effort and increasing accuracy. Additionally, implementing AI-driven insights could help users analyze spending patterns, predict future expenses, and provide personalized budgeting recommendations.

To improve accessibility, a dedicated mobile application can be developed, allowing users to manage their finances on the go. Cloud-based storage can be introduced to ensure secure, remote access to data from any device. Enhancements like advanced reporting tools, recurring transaction automation, and customizable budget trackers can make the system more user-friendly and feature-rich. Collaborative features could also be added to allow families or teams to manage shared expenses and budgets efficiently.

With these enhancements, the **Expense Tracker** can evolve into a highly versatile and comprehensive financial management tool, catering to a diverse audience and adapting to their changing needs, ultimately promoting smarter financial decisions and better money management.

APPENDIX

9. APPENDIX

Appendix I: Database Design

1. Users register Table

SL.NO	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each user
2	first_name	VARCHAR(100)	NULLABLE	Stores the user's first name
3	last_name	VARCHAR(100)	NULLABLE	Stores the user's last name
4	email_id	VARCHAR(255)	UNIQUE, NULLABLE	Unique email for user identification
5	password	VARCHAR(100)	NULLABLE	Password for user login
6	confirm_password	VARCHAR(100)	NULLABLE	Password confirmation
7	image	VARCHAR(255)	NULLABLE	Optional profile image path
8	phone	INTEGER	NULLABLE	Optional phone number
9	admin			

2. Feedback Table

SL.NO	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for feedback

2	feedback_text	TEXT	NOT NULL	Feedback content provided by the user
3	rating	INTEGER	CHECK (rating BETWEEN 1 AND 5)	Rating given by the user (1 to 5)
4	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when feedback was created
5	email	VARCHAR(255)	NOT NULL	User email linked to the feedback

3. Income Table

SL.NO	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each income entry
2	user_id	INTEGER	FOREIGN KEY(users_register.id)	Links to the user's ID
3	source	VARCHAR(100)	NULLABLE	Income source (e.g., salary, freelancing)
4	amount	DECIMAL(10,2)	NOT NULL	Amount of income
5	date	DATE	NOT NULL	Date of income receipt

4. Expense Table

SL.NO	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each expense entry

2	user_id	INTEGER	FOREIGN KEY(users_register.id)	Links to the user's ID
3	category	VARCHAR(150)	NOT NULL	Category of the expense (e.g., Food, Rent)
4	description	TEXT	NULLABLE	Description of the expense
5	amount	DECIMAL(10,2)	NOT NULL	Amount of the expense
6	date	DATE	DEFAULT CURRENT_DATE	Date of the expense

5. Bill Table

SL.NO	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each bill
2	user_id	INTEGER	FOREIGN KEY(users_register.id)	Links to the user's ID
3	category	VARCHAR(50)	NOT NULL	Category of the bill (e.g., Rent, Internet)
4	description	TEXT	NULLABLE	Description of the bill
5	amount	DECIMAL(10,2)	NOT NULL	Amount of the bill
6	due_date	DATE	NOT NULL	Due date for the bill
7	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when the bill was created
8	image	VARCHAR(255)	NULLABLE	Optional image

				related to the bill
9	is_paid	BOOLEAN	DEFAULT FALSE	Payment status (True = paid, False = unpaid)
10	razorpay_order_id	VARCHAR(255)	NULLABLE	Razorpay order ID for online payment

6. Payment Table

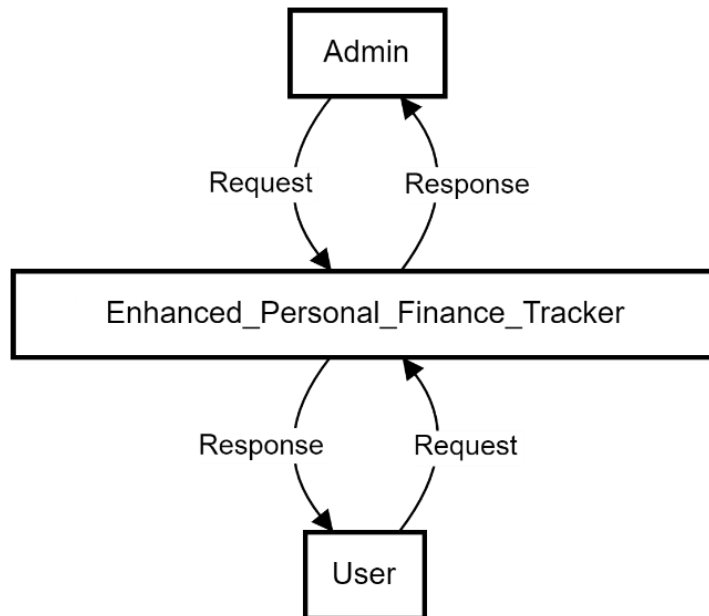
SL.NO	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each payment
2	bill_id	INTEGER	FOREIGN KEY(Bill.id)	Links to the associated bill
3	payment_id	VARCHAR(255)	NOT NULL	Unique payment ID
4	amount	DECIMAL(10,2)	NOT NULL	Payment amount
5	currency	VARCHAR(10)	DEFAULT 'INR'	Currency type
6	is_successful	BOOLEAN	DEFAULT FALSE	Payment status
7	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when the payment was created
8	updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when the payment was last updated

7.Admin Table

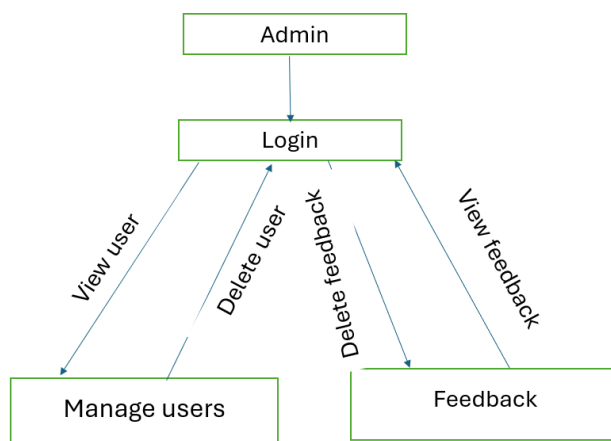
SL.N O	Column Name	Data Type	Constraints	Description
1	id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each admin entry
2	user_id	INTEGER	FOREIGN KEY(users_register.id)	Links to the user in the users_regist er table
3	role	VARCHAR(5 0)	DEFAULT 'Admin'	Specifies the role (e.g., Admin, SuperAdmin)
4	created_at	DATETIME	DEFAULT CURRENT_TIMESTA MP	Timestamp when the admin role was assigned
5	updated_a t	DATETIME	DEFAULT CURRENT_TIMESTA MP	Timestamp when the admin details were last updated

Appendix II: Dataflow Diagram

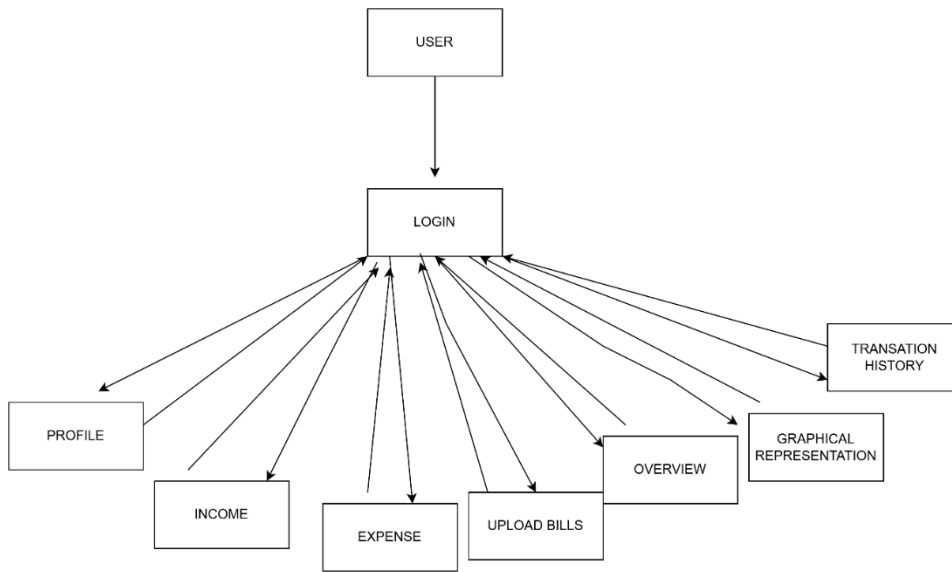
Level 0 DFD



Level 1 Admin

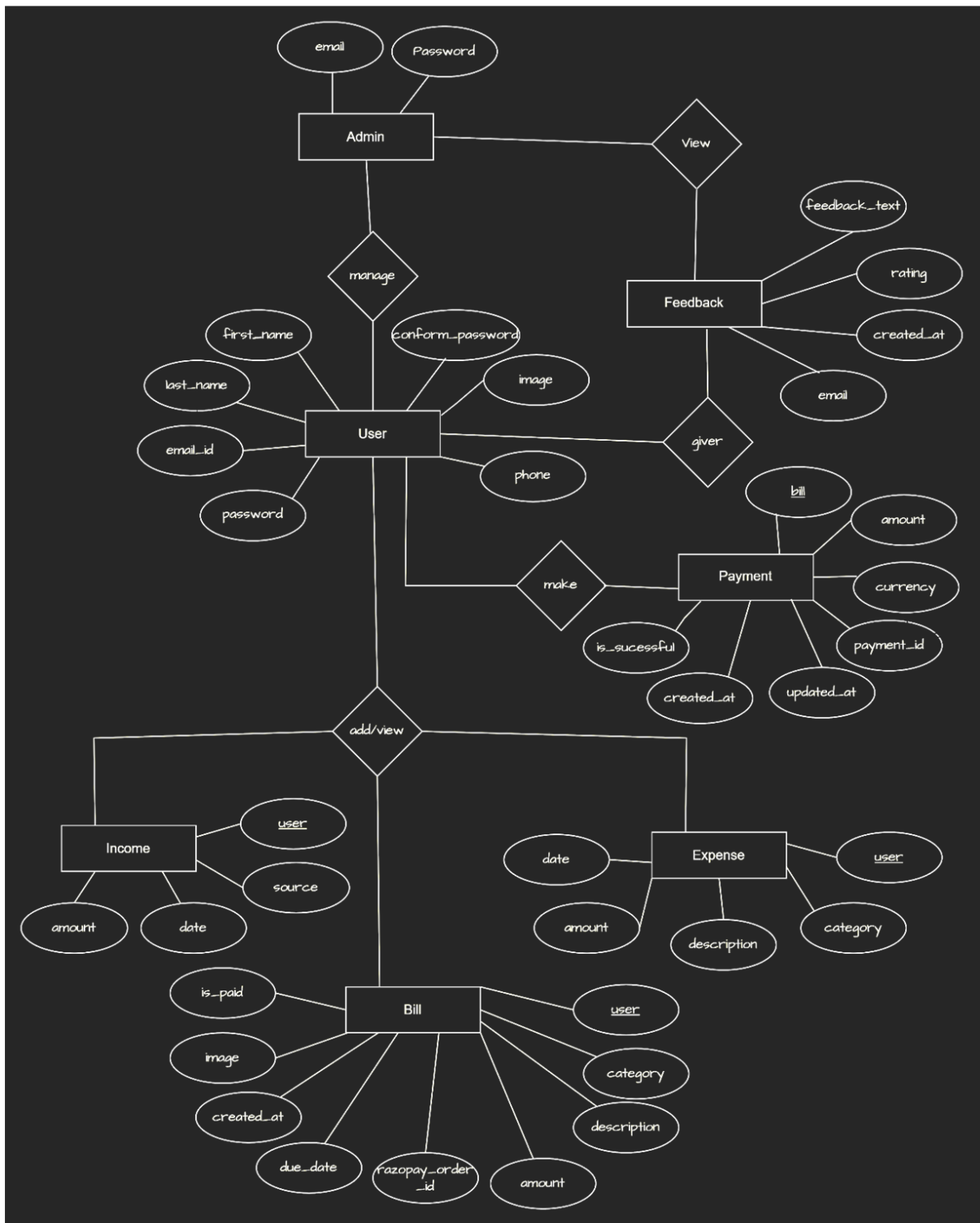


Level 2 User



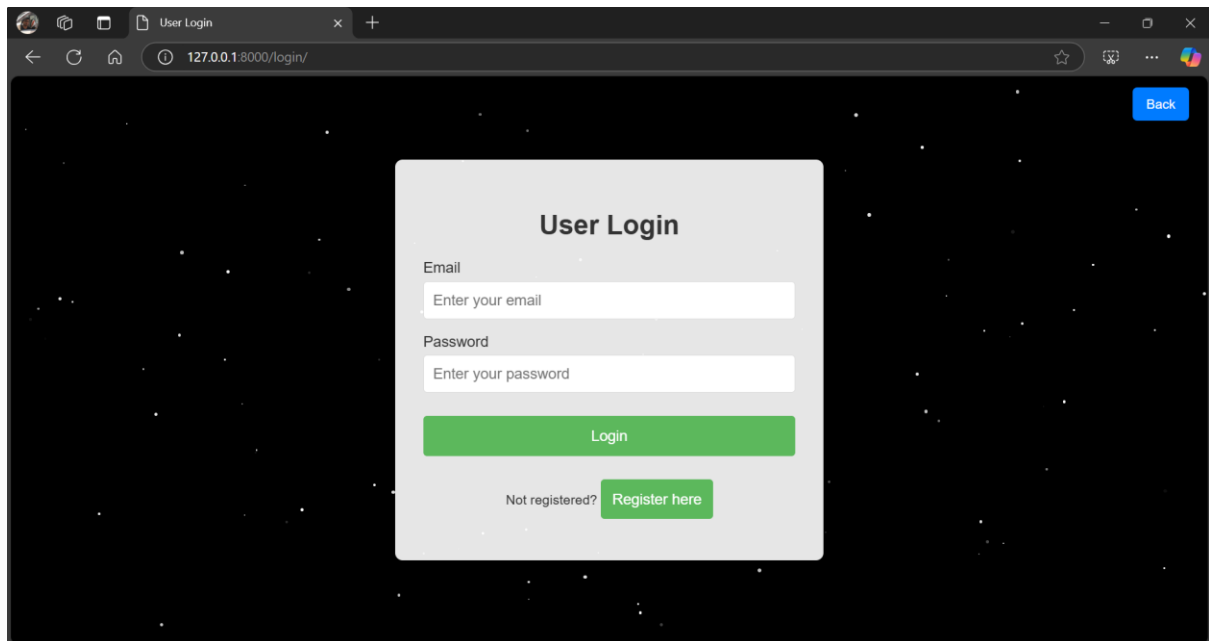
Appendix III: ER Diagram

Entity Relationship Diagram



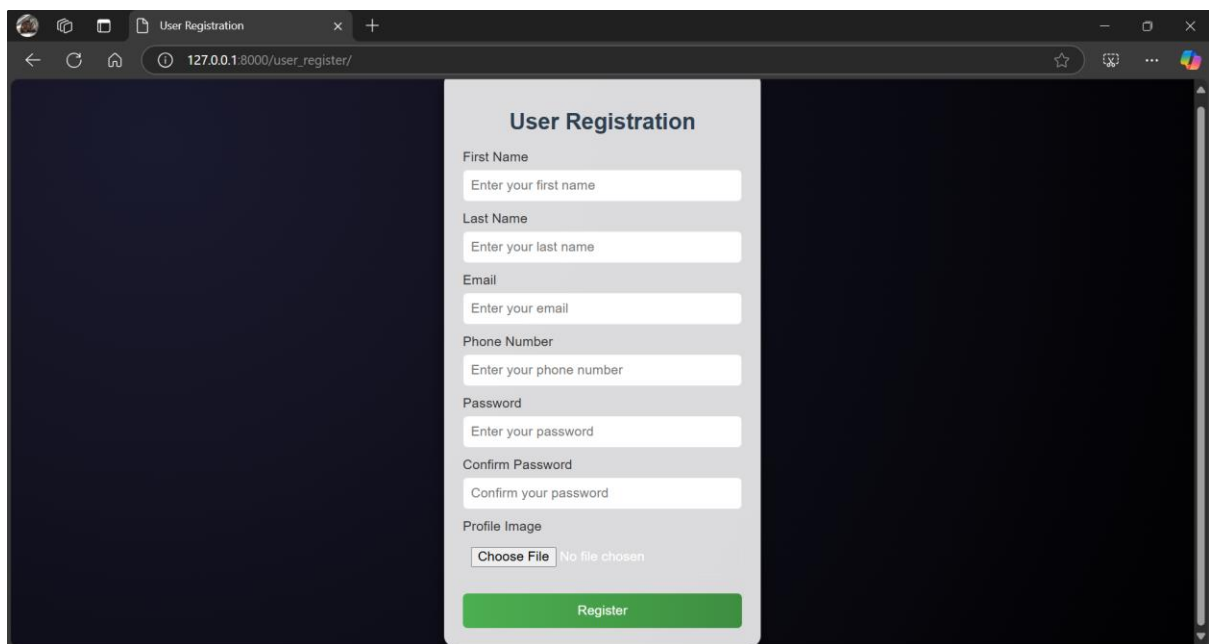
Appendix IV: Screen Layouts

SIGN IN PAGE: PUBLIC



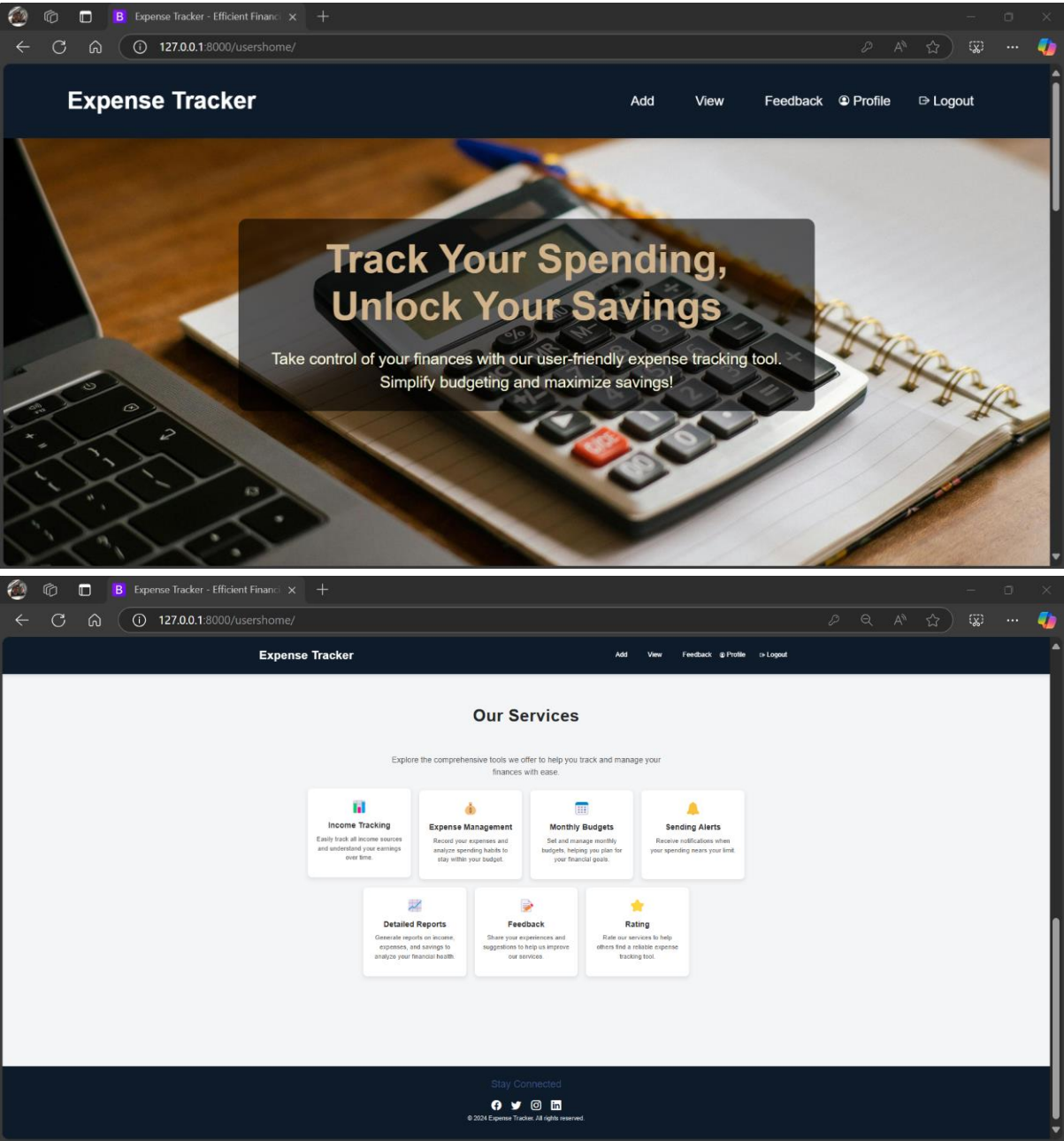
A screenshot of a web browser displaying the 'User Login' page. The browser's address bar shows '127.0.0.1:8000/login/'. The page has a dark, starry background. A light gray rectangular box is centered on the page, containing the title 'User Login'. Below the title are two input fields: 'Email' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your password'. A green 'Login' button is positioned below these fields. At the bottom of the box, the text 'Not registered?' is followed by a green 'Register here' button. A blue 'Back' button is located in the top right corner of the page.

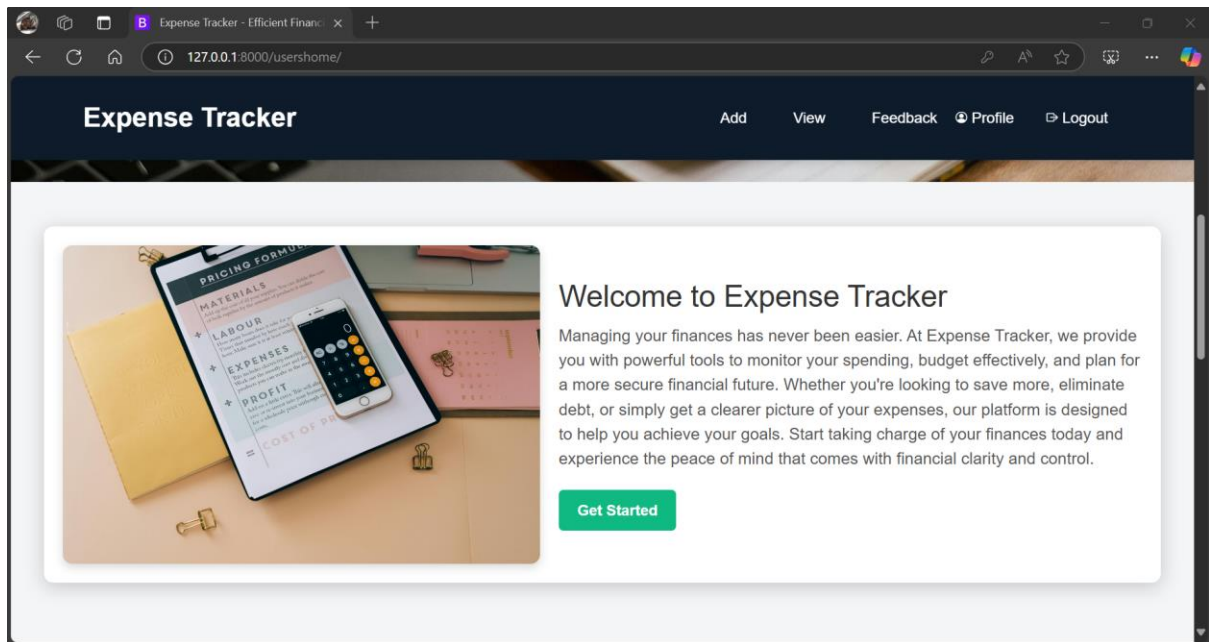
USER REGISTRATION



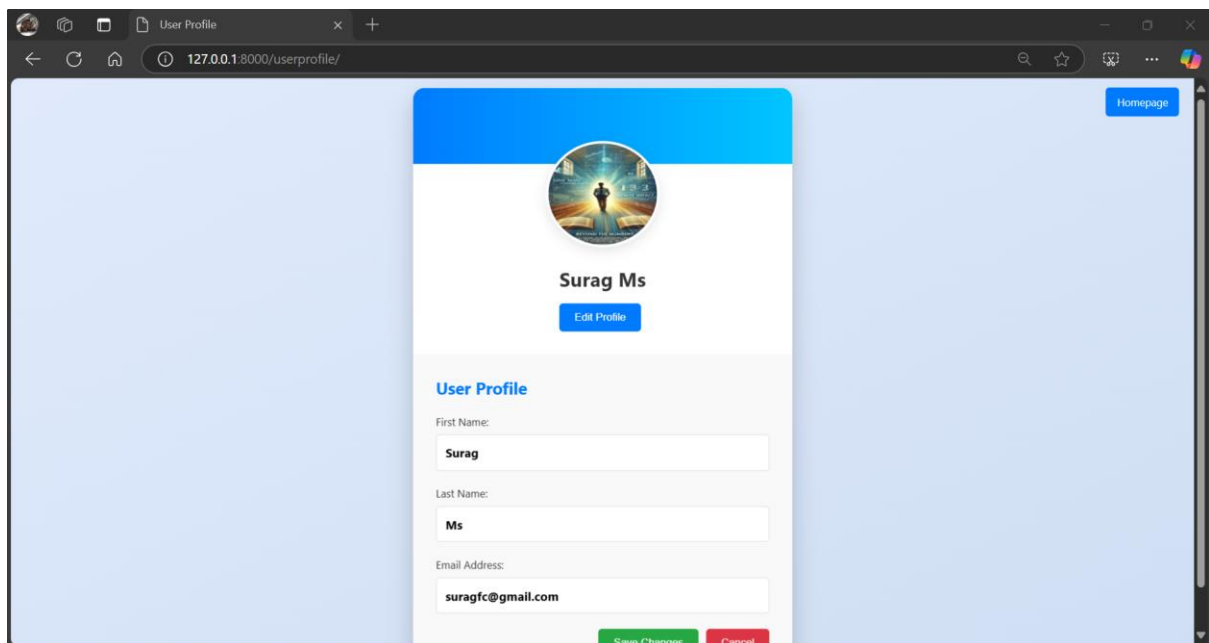
A screenshot of a web browser displaying the 'User Registration' page. The browser's address bar shows '127.0.0.1:8000/user_register/'. The page has a dark background. A light gray rectangular box is centered on the page, containing the title 'User Registration'. Below the title are several input fields: 'First Name' (placeholder: 'Enter your first name'), 'Last Name' (placeholder: 'Enter your last name'), 'Email' (placeholder: 'Enter your email'), 'Phone Number' (placeholder: 'Enter your phone number'), 'Password' (placeholder: 'Enter your password'), and 'Confirm Password' (placeholder: 'Confirm your password'). Below these fields is a 'Profile Image' section with a 'Choose File' button and the text 'No file chosen'. A green 'Register' button is at the bottom of the form box.

USERS HOME

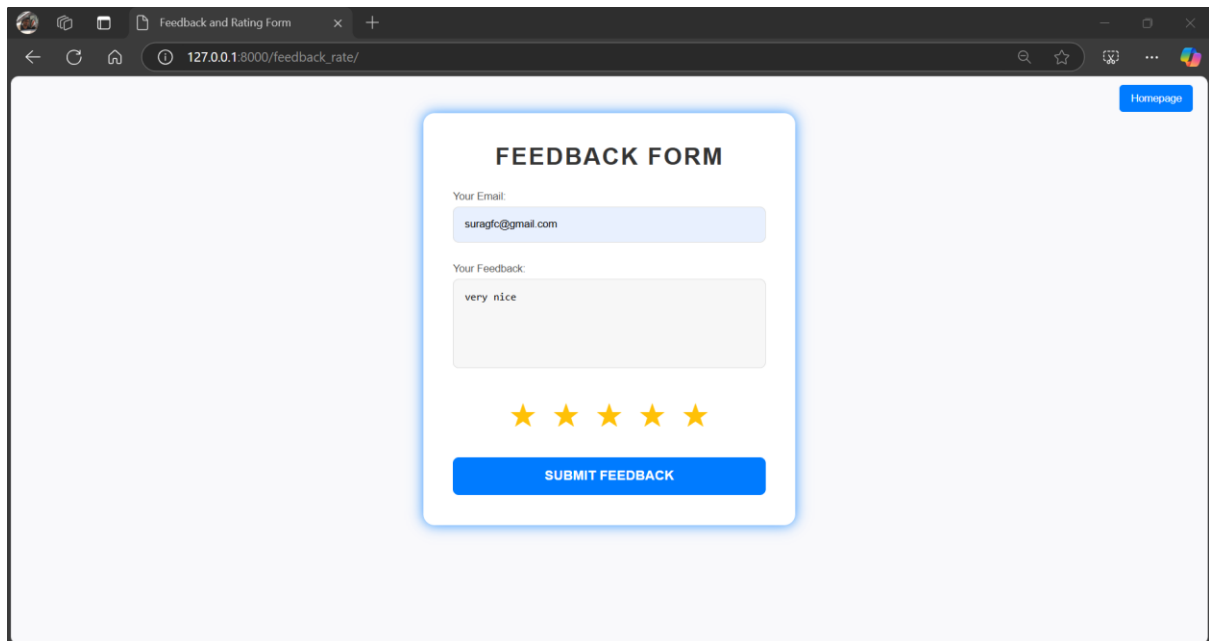




EDIT PROFILE



FEEDBACK RATE



Feedback and Rating Form

127.0.0.1:8000/feedback_rate/

Homepage

FEEDBACK FORM

Your Email:

suragfc@gmail.com

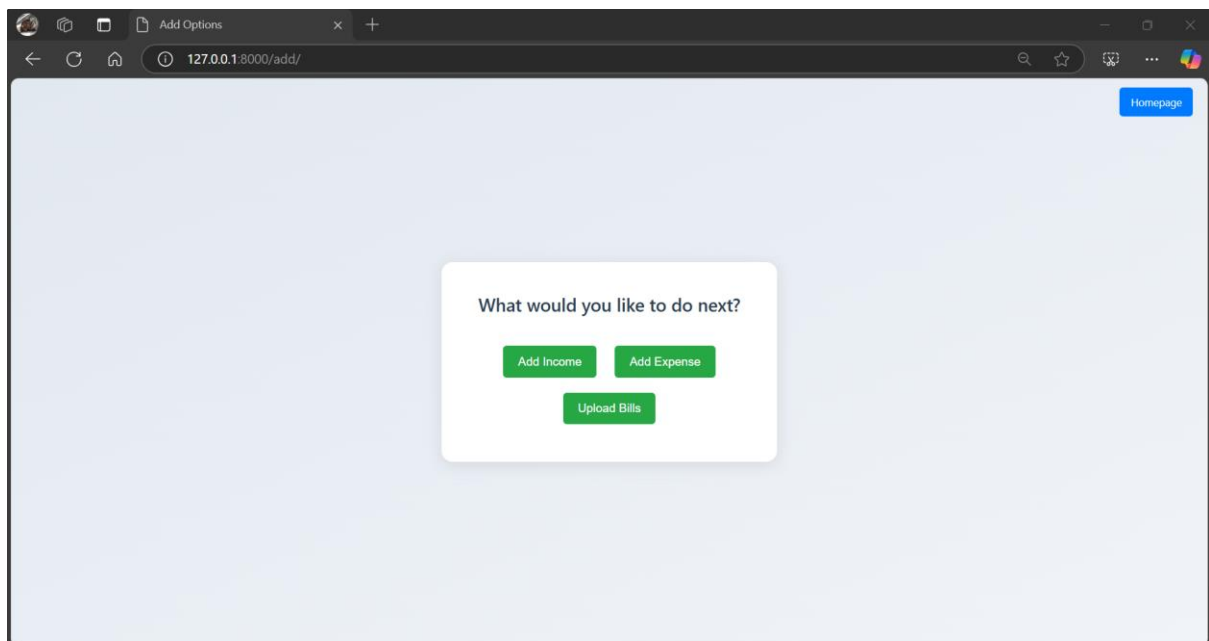
Your Feedback:

very nice

★ ★ ★ ★ ★

SUBMIT FEEDBACK

ADD FUNCTION



Add Options

127.0.0.1:8000/add/

Homepage

What would you like to do next?

Add Income Add Expense

Upload Bills

ADD INCOME

A screenshot of a web browser displaying the 'Add Income' form. The browser's address bar shows the URL '127.0.0.1:8000/add_income/33/'. The form is centered on a light blue background. It has a title 'Add Income' in blue. Below the title are three input fields: 'Source of Income:' with a placeholder 'e.g. Salary, Freelance', 'Amount:' with a placeholder 'Enter the amount', and 'Date of Income:' with a placeholder 'dd-mm-yyyy' and a calendar icon. At the bottom of the form are two buttons: a green 'Add Income' button and a red 'Reset Form' button. A 'Homepage' button is visible in the top right corner of the browser window.

ADD EXPENSE

A screenshot of a web browser displaying the 'Add Expense' form. The browser's address bar shows the URL '127.0.0.1:8000/add_expense/33/'. The form is centered on a light blue background. It has a title 'Add Expense' in black. Below the title are four input fields: 'Category' with a placeholder 'Select a category', 'Description' with a placeholder 'Enter a brief description (optional)' and a text area, 'Amount' with a placeholder, and 'Date' with a placeholder 'dd-mm-yyyy' and a calendar icon. At the bottom of the form is a green 'Submit Expense' button. A 'Homepage' button is visible in the top right corner of the browser window.

ADD BILL

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/add_bill/33/'. The page title is 'Add Bill'. In the top right corner, there is a blue button labeled 'Homepage'. The main content area features a white card with the heading 'Add a New Bill' in blue. Below the heading are several input fields: a 'Category' dropdown menu with the placeholder text 'Select a category', a 'Description' text area, an 'Amount (₹)' text input, a 'Due Date' text input with the placeholder 'dd-mm-yyyy' and a calendar icon, and an 'Upload Image (optional)' section with a 'Choose File' button and the text 'No file chosen'. At the bottom of the card is a prominent blue button labeled 'Add Bill'.

VIEW

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/view/'. The page title is 'View Options'. In the top right corner, there is a blue button labeled 'Homepage'. The main content area features a white card with the heading 'What would you like to view?'. Below the heading are six buttons arranged in a 3x2 grid. The buttons are: 'View Income' (grey), 'View Expense' (green), 'Graphical representation' (green with a dropdown arrow), 'Overview' (grey), 'Transaction History' (grey), and 'View Bill' (green).

VIEW INCOME

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/view_income/'. The page title is 'View Monthly Income'. The main content area features a card titled 'Your Income by Month' for 'November 2024'. The card contains a table with income entries and a 'Delete' button in the top right corner.

Date	Source	Amount (₹)	Delete
Nov. 15, 2024	Youtube income	₹40000.00	Delete
Nov. 17, 2024	Salary Expenses	₹200000.00	Delete

VIEW EXPENSE

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/view_expense/'. The page title is 'View Monthly Expenses'. The main content area features a card titled 'Your Expenses by Month' for 'November 2024'. The card contains a table with expense entries and a 'Delete' button in the top right corner.

Date	Category	Description	Amount (₹)	Delete
Nov. 28, 2024	Entertainment	Netflix prime	₹487.00	Delete
Nov. 27, 2024	Entertainment	Movie ticket	₹265.00	Delete
Nov. 16, 2024	Transport	Patrol for car	₹3500.00	Delete
Nov. 9, 2024	Other	For recharge mobile jio	₹859.00	Delete

VIEW ALL TRANSNCTIONS

All Transactions

HomepageBack

All Transactions

Income Transactions

Date	Source	Amount
Nov. 17, 2024	Salary Expenses	+200000.00
Nov. 15, 2024	Youtube income	+40000.00
Total Income:		+240000.00

Expense Transactions

Date	Description	Amount
Nov. 28, 2024	Netflix prime	-487.00
Nov. 27, 2024	Movie ticket	-265.00
Nov. 16, 2024	Patrol for car	-3500.00
Nov. 9, 2024	For recharge mobile jio	-859.00
Total Expense:		-5111.00

VIEW BILLS

Your Bills

127.0.0.1:8000/bill/

HomepageBack

127.0.0.1:8000 says

Reminder: Your bill with due date Nov. 19, 2024 is due in 2 day(s)!

OK

Bill ID: 14

Category: Electricity

Description: Home

Amount: ₹870.00

Due Date: Nov. 19, 2024

Status: Unpaid



Bill ID: 15

Category: Internet

Description: Work from home

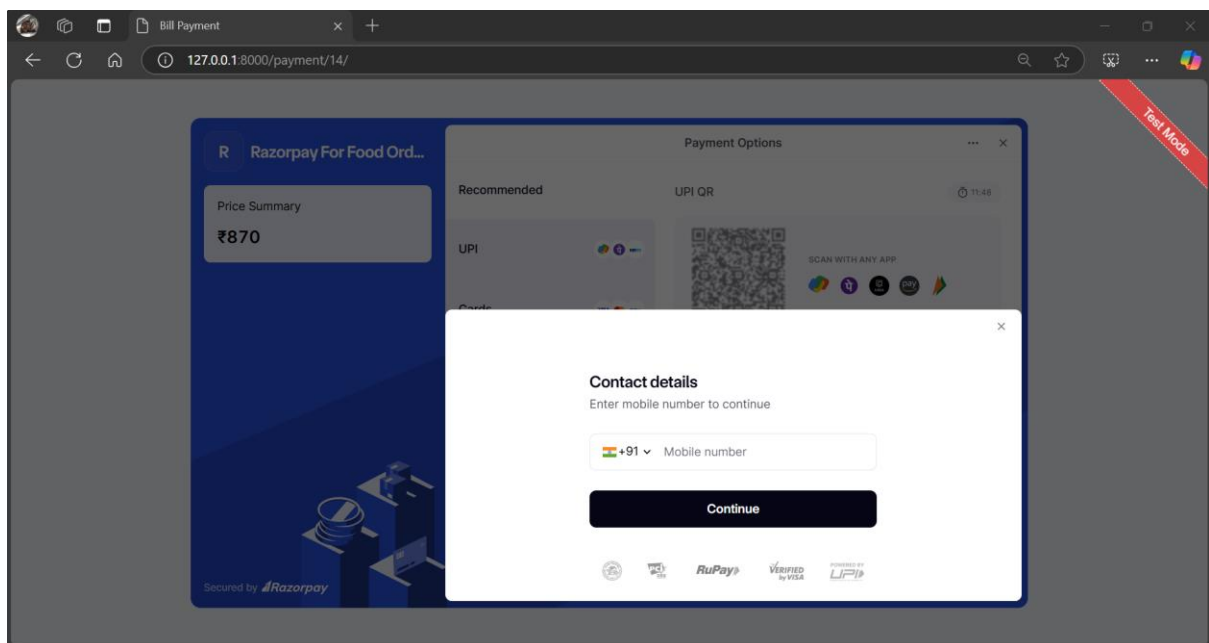
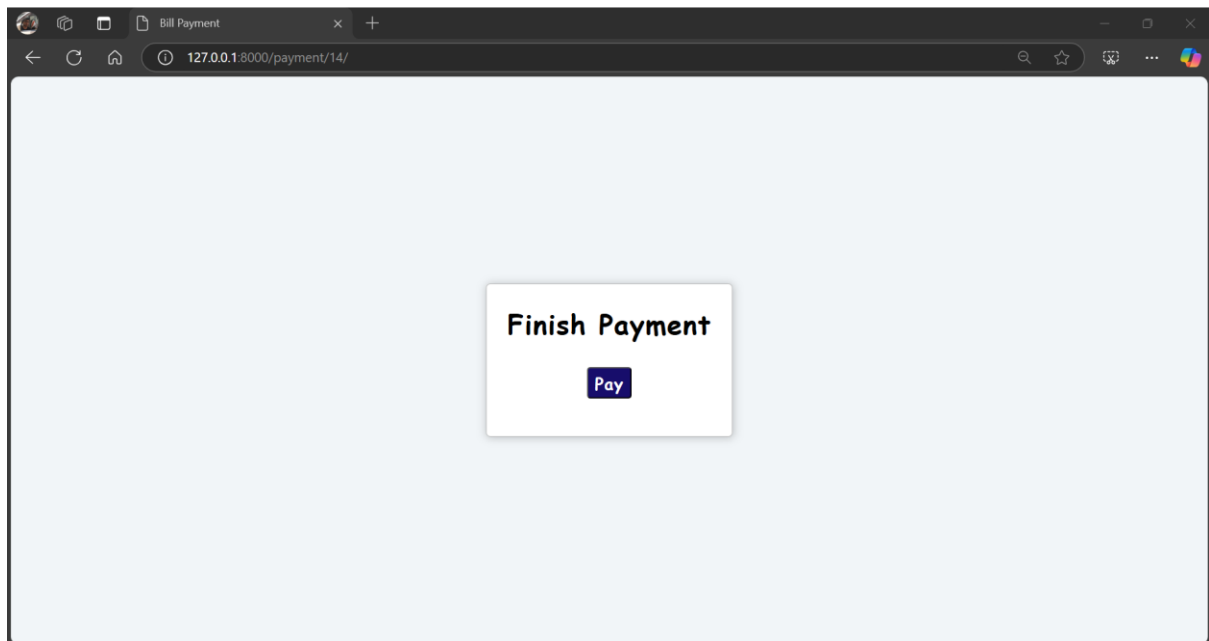
Amount: ₹380.00

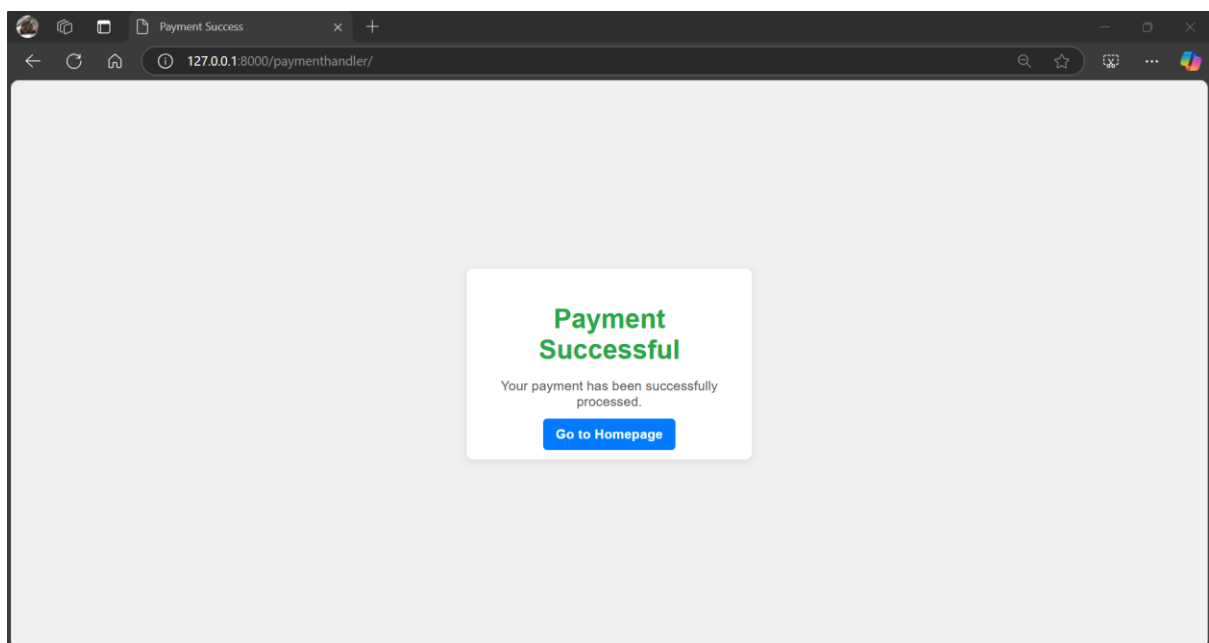
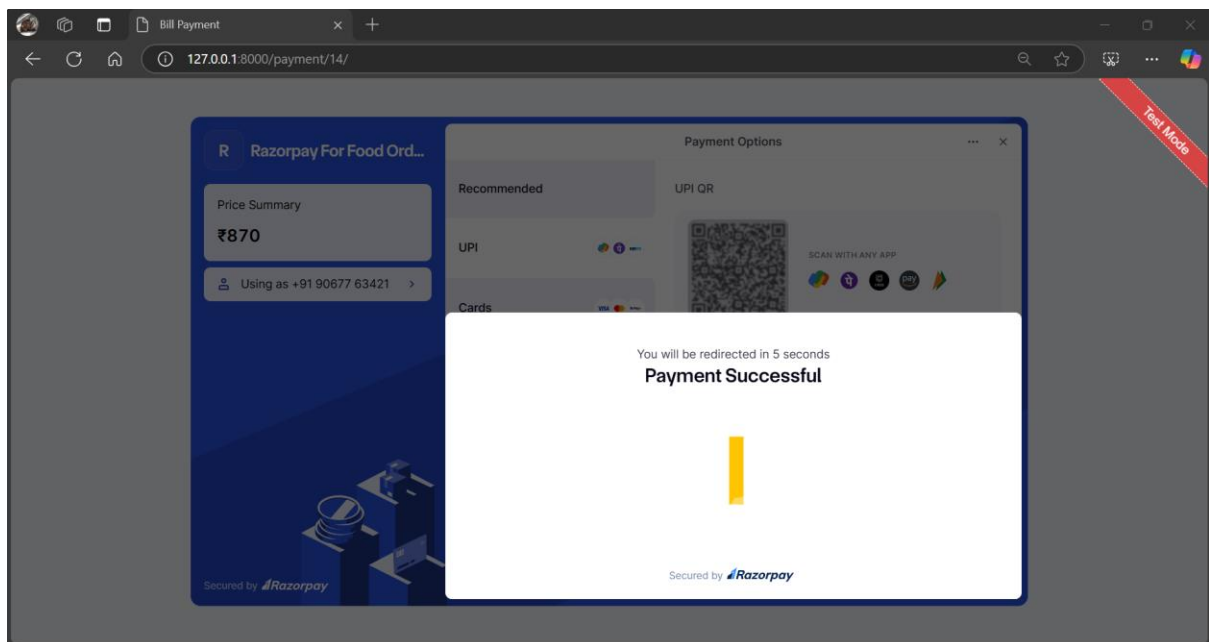
Due Date: Nov. 20, 2024

Status: Unpaid

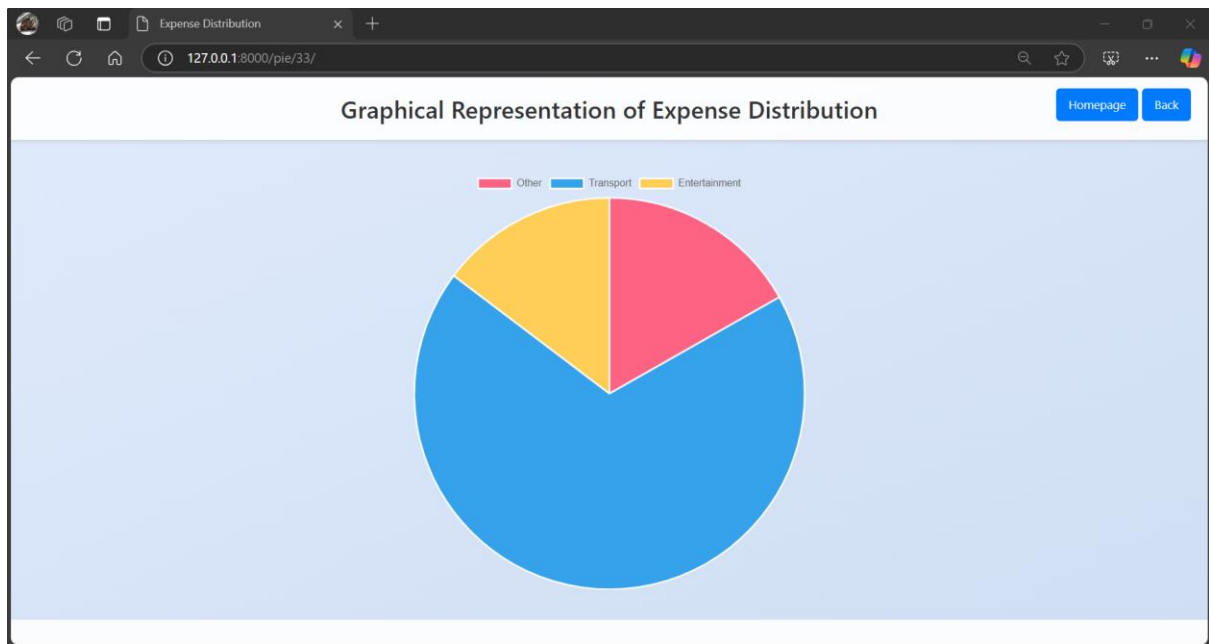


PAYMENT OPTIONS

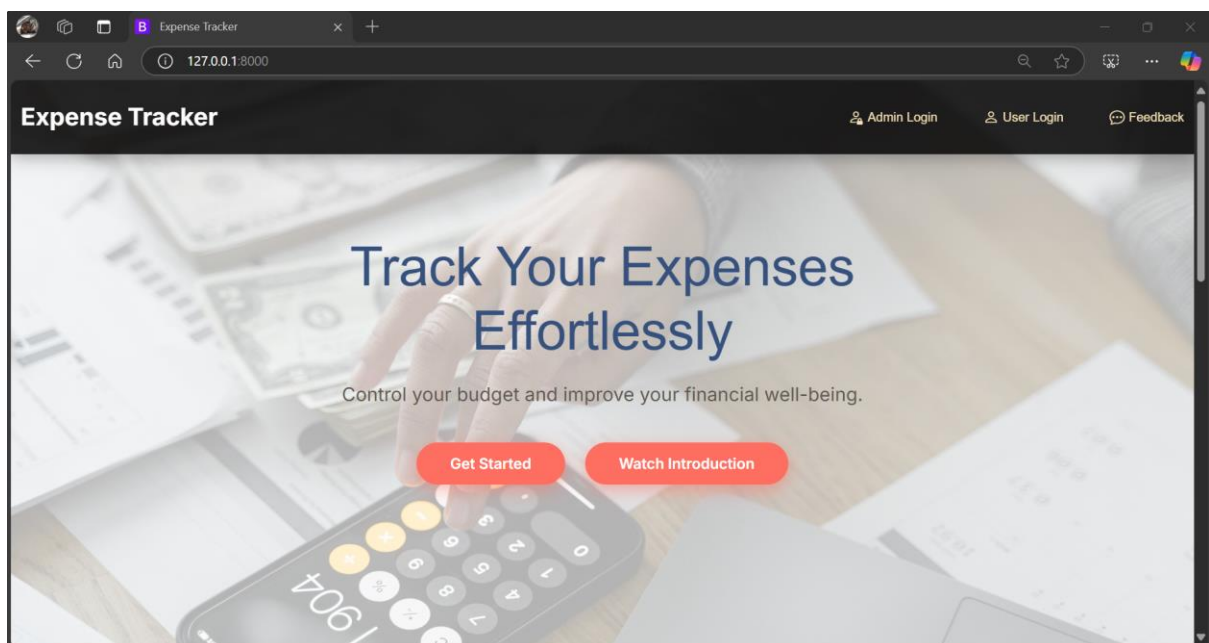




GRAPHICAL RESPRESENTATIONS



INDEX PAGE



10.BIBLIOGRAPHY

The following references and resources were utilized in the development of the **Expense Tracker** system and its associated documentation:

1. Books and Study Material:

- Sommerville, Ian. *Software Engineering*, 10th Edition, Pearson Education.
- Pressman, Roger S. *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill.

2. Web Resources:

- Django Official Documentation: <https://www.djangoproject.com>
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- Python Official Documentation: <https://docs.python.org>

3. Tools and Technologies:

- Django Framework for backend development.
- SQLite3 for database management.
- Visual Studio Code for development and debugging.
- HTML, CSS, JavaScript for frontend design.

4. Online Tutorials and Guides:

- TutorialsPoint: Comprehensive guides on Python and Django.
- W3Schools: Resources for web development technologies like HTML, CSS, and JavaScript.

5. Research Papers and Articles:

- Articles on software testing methodologies, feasibility studies, and system design principles.
- Papers on the importance of financial management systems in personal finance.

6. Community Support:

- Stack Overflow: Technical assistance for debugging and best practices in Django.

These references provided valuable insights and tools, contributing to the successful development and documentation of the **Expense Tracker** system.